



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de **HONORIS UNITED UNIVERSITIES**

Projet de fin d'année

3^e année en Ingénierie Informatique et Réseaux

IoT Network-Flow Intrusion Detection

Réalisé par :

Zakaria El Khaldi

Encadrant académique :

Dr. Anass Mancourbillah

Année universitaire : 2026–2027

Dédicaces

Ce modeste travail est avant tout le fruit de l'amour et du soutien de ceux qui comptent le plus pour moi.

À ma chère famille, qui a toujours placé mon éducation et mon épanouissement avant tout ; Merci pour chaque sacrifice fait dans l'ombre, pour votre patience à toute épreuve et pour la confiance que vous m'accordez depuis toujours. Que la vie vous rende au centuple ce que vous m'avez offert, et vous préserve longtemps auprès de moi.

À mes amis, compagnons de toujours ; Merci pour votre écoute, votre bonne humeur et vos précieux conseils dans les moments de doute. Ce travail est aussi le vôtre : je vous souhaite à mon tour bonheur, santé et réussite dans vos propres chemins.

Remerciements

Nous adressons nos remerciements à l'équipe pédagogique de l'EMSI pour le cadre de formation qui a rendu possible la réalisation de ce projet de fin d'année. Les enseignements consacrés au génie logiciel, aux réseaux, à la sécurité informatique et à l'apprentissage automatique ont fourni les bases nécessaires à sa conception.

Nous remercions également les auteurs des travaux scientifiques et des outils libres cités dans ce rapport. Leurs publications, documentations et logiciels ont permis de construire une démarche reproductible et de confronter les choix du prototype à des références vérifiables.

Enfin, nous exprimons notre reconnaissance à toutes les personnes ayant relu le travail ou formulé des observations utiles à son amélioration.

Résumé

Ce projet étudie la détection d'intrusions à partir de flux réseau IoT et la consultation des alertes dans une application de supervision. Il utilise RT-IoT2022, dont les 123 117 lignes sont contrôlées puis nettoyées des conflits d'étiquettes et des doublons. Quatre familles de modèles sont comparées sur des partitions d'entraînement, de validation et de test, avec trois graines aléatoires. La sélection retient un gradient boosting par histogrammes pour distinguer les attaques du trafic normal, puis une forêt aléatoire pour identifier leur famille.

Une API FastAPI exécute les modèles et enregistre les observations, les prédictions et les alertes. L'ingestion persistante permet de reprendre les traitements en échec sans créer de doublons. L'interface React donne accès aux résultats, aux versions des modèles et à l'historique des qualifications de l'analyste. Les alertes de signature du laboratoire Suricata y sont identifiées par leur source.

Sur le test interne associé à la graine 42, la cascade obtient une macro-F1 de 0,9520; la moyenne sur les trois graines est de 0,9686. Les classes rares restent fragiles : le test ne contient que six flux NMAP_FIN_SCAN et sept flux Metasploit_Brute_Force_SSH. L'étude fournit ainsi une évaluation interne et un prototype d'investigation. Une collecte NFStream étiquetée et une évaluation sur un réseau externe restent nécessaires pour en étudier le transfert.

Abstract

This project studies intrusion detection from IoT network flows and the review of alerts in a monitoring application. It uses RT-IoT2022, checking its 123,117 rows and removing conflicting labels and duplicate observations. Four model families are compared on training, validation, and test partitions using three random seeds. The selection procedure chooses histogram-based gradient boosting to distinguish attacks from normal traffic, followed by a random forest to identify attack families.

A FastAPI service runs the models and stores observations, predictions, and alerts. A persistent ingestion queue supports retries without creating duplicate records. The React interface provides access to results, model versions, and the history of analyst dispositions. Signature alerts from the Suricata laboratory are identified by their source.

On the internal test split for seed 42, the cascade achieves a macro-F1 of 0.9520; the mean across the three seeds is 0.9686. Results for rare classes remain fragile: the test contains only six NMAP_FIN_SCAN flows and seven Metasploit_Brute_Force_SSH flows. The study provides an internal evaluation and an investigation prototype. A labelled NFStream collection and evaluation on an external network are still needed to assess transfer.

Table des matières

Introduction générale	1
1 Cadre scientifique et problématique de recherche	2
1.1 Introduction	2
1.2 Contexte scientifique	2
1.2.1 Détection d'intrusions dans les environnements IoT	2
1.2.2 Jeu de données RT-IoT2022	4
1.3 Analyse des approches existantes	5
1.3.1 Détection par signatures et détection statistique	5
1.3.2 Classification supervisée des flux	5
1.3.3 Limites des validations usuelles	6
1.4 Problématique et questions de recherche	6
1.5 Solution proposée et périmètre	7
1.6 Démarche de recherche	7
1.6.1 Mode de développement	7
1.6.2 Phases de recherche	7
1.7 Conclusion	8
2 Spécification des besoins	9
2.1 Introduction	9
2.2 Besoins fonctionnels	9
2.3 Besoins non fonctionnels	10
2.4 Présentation des cas d'utilisation	10
2.5 Présentation des acteurs	10
2.6 Diagramme global des cas d'utilisation	11
2.7 Description des cas d'utilisation	12

2.7.1	UC-02 — Soumettre des observations de flux	12
2.7.2	UC-03 — Transmettre des événements Suricata	12
2.7.3	UC-04 — Contrôler le rejeu du jeu de données	13
2.7.4	UC-05 — Examiner et qualifier une alerte	13
2.8	Conclusion	14
3	Conception du système	15
3.1	Introduction	15
3.2	Modélisation statique	15
3.2.1	Modèle du domaine	15
3.3	Architecture de l'application	20
3.4	Modélisation dynamique	22
3.4.1	Activité de l'opérateur	22
3.4.2	Communication pendant l'investigation	23
3.4.3	Ingestion durable des observations	23
3.4.4	Prédiction synchrone	25
3.4.5	Investigation des alertes	26
3.4.6	Surveillance de la santé des modèles	27
3.4.7	Authentification, rejeu et capteur	27
3.5	Conclusion	27
4	Réalisation du système	28
4.1	Introduction	28
4.2	Environnement de développement	28
4.2.1	Environnement matériel	28
4.2.2	Environnement logiciel	28
4.3	Organisation de la réalisation	30
4.4	Principaux choix d'implémentation	30
4.4.1	Du modèle entraîné à la prédiction servie	30
4.4.2	Ingestion durable et traitement des erreurs	30

4.4.3	Interface, contrôle d'accès et provenance	31
4.4.4	Exécution reproductible	31
4.5	Principales interfaces graphiques	32
4.5.1	Supervision opérationnelle	32
4.5.2	Investigation des alertes	33
4.5.3	Analyse des modèles	34
4.5.4	Laboratoire d'observations	35
4.6	Validation technique	35
4.7	Difficultés rencontrées et solutions apportées	36
4.7.1	Comprendre les caractéristiques du jeu de données	36
4.7.2	Savoir quand passer des essais à l'application	36
4.7.3	Rendre le tableau de bord compréhensible	36
4.8	Conclusion	36
5	Résultats expérimentaux	37
5.1	Données et fichiers de référence	37
5.2	Qualité, nettoyage et déséquilibre	37
5.3	Protocole expérimental	38
5.4	Comparaison des classifieurs	38
5.5	Expérience sur le déséquilibre	39
5.6	Erreurs des modèles retenus et de la cascade	40
5.7	Calibration, seuil et coût opérationnel	40
5.8	Menaces à la validité	41
5.9	Conclusion	41
	Conclusion générale	42
	Bibliographie et webographie	43

Liste des figures

1.1	Architecture de référence d'un IDPS réseau passif	3
1.2	Exemple d'architecture de capture IoT	5
1.3	Chronologie indicative des phases de développement	8
2.1	Cas d'utilisation du système de détection	11
3.1	Diagramme de classes du domaine principal	16
3.2	Classes associées à l'ingestion durable	17
3.3	Classes associées au suivi de l'état des modèles	18
3.4	Services et opérations principales	19
3.5	Architecture logicielle par composants	20
3.6	Déploiement du mode de démonstration Docker	21
3.7	Activité principale de l'opérateur	22
3.8	Communication entre objets lors de l'investigation d'une alerte	23
3.9	Séquence d'ingestion durable et de diffusion des résultats	24
3.10	Machine à états d'un travail d'ingestion	24
3.11	Séquence de prédiction synchrone	25
3.12	Séquence d'investigation et de qualification d'une alerte	26
3.13	Séquence de consultation de la santé d'un modèle	27
4.1	Espace principal de supervision du prototype	32
4.2	Espace d'investigation des alertes	33
4.3	Espace d'analyse et de traçabilité des modèles	34
4.4	Laboratoire de validation et de soumission d'observations	35
5.1	Comparaison des candidats dans le notebook d'analyse	39

Liste des tableaux

1.1	Résultats de l’audit local du fichier RT-IoT2022 contrôlé	4
2.1	Besoins fonctionnels du prototype	9
2.2	Besoins non fonctionnels et critères observables	10
2.3	Description du cas UC-02 — Soumettre des observations de flux	12
2.4	Description du cas UC-03 — Transmettre des événements Suricata	12
2.5	Description du cas UC-04 — Contrôler le rejeu du jeu de données	13
2.6	Description du cas UC-05 — Examiner et qualifier une alerte	13
4.1	Environnement logiciel et rôle des principaux outils	29
5.1	Comparaison du détecteur binaire sur les trois graines	38
5.2	Comparaison du classifieur des familles d’attaque	39
5.3	Comparaison contrôlée des traitements du déséquilibre	39
5.4	Classes les plus fragiles dans la cascade, graine 42	40

Introduction générale

Une alerte de sécurité doit pouvoir être examinée. Pour un opérateur, connaître la classe prédite ne suffit pas : il faut retrouver l’observation analysée, le modèle utilisé et les éléments qui ont conduit au signalement. Ce projet aborde la détection d’intrusions IoT à partir de ce besoin, en reliant l’évaluation de modèles sur des flux enregistrés à une application qui conserve et présente leurs résultats.

Le travail s’appuie sur RT-IoT2022, un jeu de données public de communications normales et d’attaques IoT (Sharmila and Nagapadma, 2023a). Un premier modèle distingue le trafic normal des attaques; un second attribue une famille aux flux signalés. La préparation des données, la comparaison des modèles et l’analyse des erreurs précèdent leur intégration dans une API et une interface d’investigation. L’application permet aussi de rejouer les observations du jeu de données et de consulter séparément les alertes de signature d’un laboratoire Suricata.

La question est donc la suivante : comment évaluer cette cascade de classification et rendre ses décisions consultables dans un prototype IoT? L’évaluation porte sur des partitions de RT-IoT2022. Elle ne mesure pas encore l’efficacité du modèle sur le trafic d’un autre réseau; cette distinction fixe la portée des résultats présentés.

Le rapport comprend cinq chapitres. Le premier présente les travaux de référence et les caractéristiques du jeu de données. Le deuxième précise les besoins de l’opérateur et des producteurs d’observations. Le troisième décrit les données, les composants et leurs interactions. Le quatrième explique la réalisation et les vérifications techniques. Le cinquième compare les modèles, analyse les erreurs de la cascade et discute les limites de l’expérience.

CHAPITRE 1

Cadre scientifique et problématique de recherche

1.1 Introduction

Le prototype *IoT Network-Flow Intrusion Detection* classe des observations de flux et conserve les résultats pour leur examen. Deux questions guident les choix scientifiques : ce que les variables de RT-IoT2022 permettent de mesurer, et ce qu’une évaluation sur ce seul jeu de données permet de conclure.

1.2 Contexte scientifique

1.2.1 Détection d’intrusions dans les environnements IoT

Un système de détection et de prévention d’intrusions observe des événements sur un hôte ou un réseau afin d’identifier des activités susceptibles de compromettre la confidentialité, l’intégrité ou la disponibilité des ressources (Scarfone and Mell, 2007). Dans un environnement IoT, cette fonction est rendue plus difficile par l’hétérogénéité des appareils, des protocoles et des capacités matérielles. La littérature distingue notamment les méthodes selon la source des données, la stratégie de détection et le positionnement du détecteur dans l’architecture IoT (Zarpelão et al., 2017).

Le présent projet se limite aux observations agrégées par flux réseau. Une observation décrit les deux sens d’une communication au moyen de compteurs, de durées, de statistiques de paquets et d’attributs protocolaires. Cette représentation est plus compacte qu’une analyse du contenu de chaque paquet, mais sa valeur dépend directement de la définition et de la reproductibilité de l’extracteur. Des outils tels que NFStream permettent de construire des mesures de flux programmables; ils ne garantissent toutefois pas, à eux seuls, l’équivalence avec les variables d’un jeu de données produit par un autre extracteur (Aouini and Pekar, 2022).

Dans une architecture passive, les capteurs reçoivent une copie du trafic. La figure 1.1, issue du NIST, situe les points de copie et le réseau de gestion des capteurs. Elle sert de référence pour ce mode d’observation.

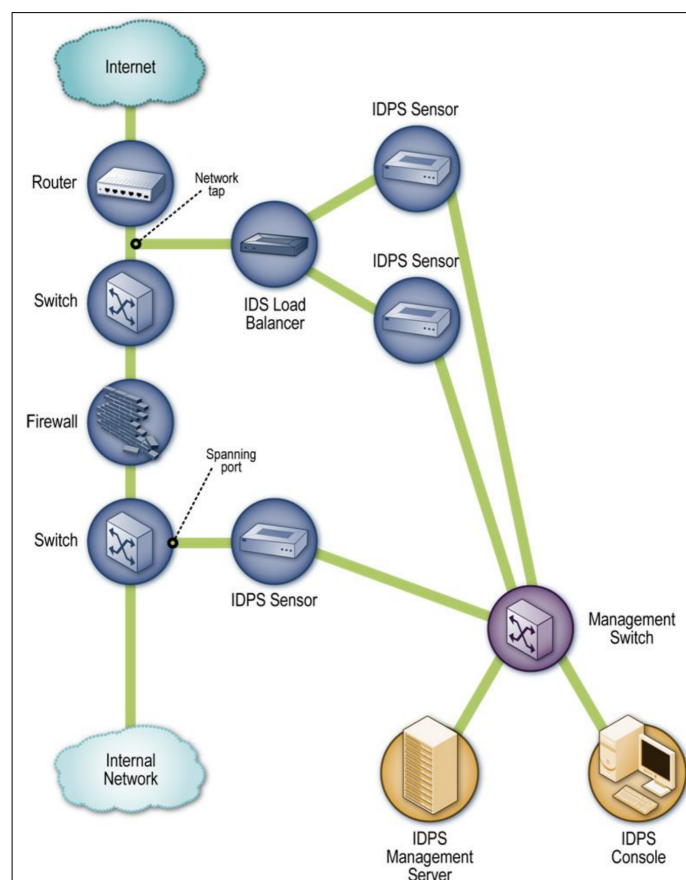


Figure 1.1 – Architecture de référence d’un IDPS réseau passif. Source: figure 4-3 de Scarfone and Mell (2007).

1.2.2 Jeu de données RT-IoT2022

RT-IoT2022 a été choisi comme support expérimental. La notice UCI le présente comme un jeu de données tabulaire multivarié consacré au trafic IoT et publie 123 117 observations comportant 83 variables (Sharmila and Nagapadma, 2023a). Le travail associé étudie également un autoencodeur quantifié destiné à la détection d'anomalies sur des dispositifs aux ressources contraintes (Sharmila and Nagapadma, 2023b).

L'analyse porte sur le fichier CSV officiel, dont l'intégrité est vérifiée par une empreinte SHA-256. L'examen de ce fichier confirme 123 117 lignes, 83 variables explicatives et douze étiquettes observées: trois classes de trafic normal et neuf familles d'attaque. Il relève également 5 202 lignes excédentaires à caractéristiques identiques et une forte concentration de la classe DOS_SYN_Hping. Ces propriétés imposent une déduplication avant la séparation des données et une lecture prudente des scores agrégés.

Tableau 1.1 – Résultats de l'audit local du fichier RT-IoT2022 contrôlé

Propriété vérifiée	Valeur observée
Observations sources	123 117
Variables utilisées par les modèles	83
Étiquettes observées	12
Observations normales	12 507
Observations d'attaque	110 610
Lignes excédentaires à caractéristiques identiques	5 202

La notice UCI indique une extraction avec Zeek et un module Flowmeter (Sharmila and Nagapadma, 2023a). Elle ne fournit toutefois pas les paramètres nécessaires pour reproduire ici les valeurs des 83 variables. Pour NFStream, les temporisations de flux et le niveau de comptage des octets font partie des paramètres d'extraction (NFStream contributors, s.d.). La correspondance des noms de variables ne suffit donc pas à établir leur équivalence numérique. Le CSV ne fournit pas non plus d'identifiant fiable de session, d'appareil ou d'horodatage permettant une partition groupée ou chronologique.

Pour préparer une autre collecte, le NIST décrit un environnement d'observation d'appareils filaires et sans fil (Watrobski et al., 2025). La figure 1.2 en montre l'architecture; elle ne décrit pas l'environnement de collecte de RT-IoT2022.

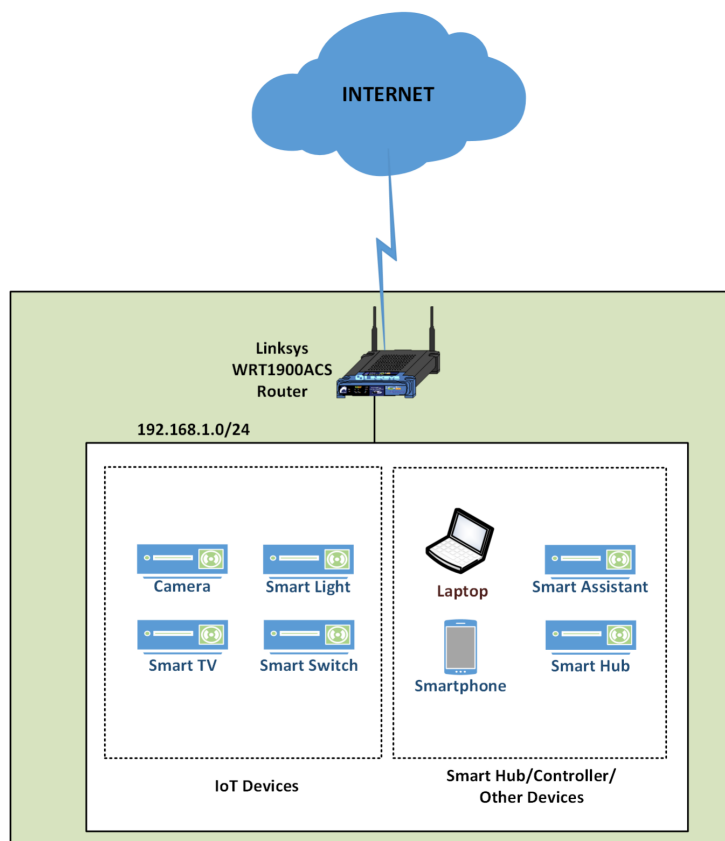


Figure 1.2 – Exemple d’architecture de capture IoT. Source: figure 4 de Watrobski et al. (2025).

1.3 Analyse des approches existantes

1.3.1 Détection par signatures et détection statistique

Une signature recherche un motif défini par une règle; la qualité de la détection dépend donc de la couverture de ces règles. Une méthode de détection d’anomalies compare l’activité observée à un comportement de référence (Scarfone and Mell, 2007). La classification supervisée adoptée ici apprend, quant à elle, à distinguer les étiquettes présentes dans les données d’entraînement. Elle n’établit pas une capacité à reconnaître des attaques absentes de ces données.

Le prototype conserve deux sources de signalement : les prédictions de la cascade et les correspondances de règles Suricata. L’opérateur peut ainsi retrouver le modèle ou la signature à l’origine d’une alerte.

1.3.2 Classification supervisée des flux

Le banc d’essai du projet compare quatre familles disponibles dans Scikit-learn (Pedregosa et al., 2011): régression logistique, arbre de décision, forêt aléatoire et gradient boosting par histogrammes. Les forêts aléatoires combinent plusieurs arbres construits avec de l’aléa afin de

réduire la variance (Breiman, 2001); le gradient boosting construit progressivement un modèle additif optimisant une fonction de perte (Friedman, 2001).

L'architecture retenue est une cascade à deux étages. Le premier classifieur sépare les flux normaux des attaques. Le second n'est sollicité que pour les flux classés comme attaques et attribue alors une famille. Cette séparation reflète le chemin d'inférence réel: une attaque manquée par le premier étage ne peut recevoir aucune famille au second. Par conséquent, l'évaluation doit publier à la fois les résultats de chaque étage et ceux de la cascade complète.

1.3.3 Limites des validations usuelles

Une erreur de protocole peut produire un score optimiste sans améliorer le système réel. La fuite de données apparaît lorsqu'une information indisponible au moment de la prédiction influence l'entraînement ou la sélection du modèle (Kaufman et al., 2012). Dans le cas présent, le prétraitement doit donc être ajusté uniquement sur l'ensemble d'entraînement, et les doublons exacts doivent être traités avant la partition.

La généralisation entre environnements est une seconde menace. Une étude inter-jeux de données rapporte que des classifieurs presque parfaits dans leur jeu d'origine peuvent perdre fortement en performance lorsqu'ils sont appliqués à une autre collecte (Cantone et al., 2024). Ce résultat ne mesure pas directement RT-IoT2022, mais il interdit d'assimiler une validation aléatoire sur un seul fichier à une validation sur un réseau de déploiement.

Enfin, l'exactitude seule est insuffisante lorsque les classes sont déséquilibrées. Le projet privilégie donc la macro-F1, les matrices de confusion, le support et le rappel par classe, ainsi que le taux de faux positifs. Ces mesures permettent d'examiner à la fois les attaques manquées et les fausses alertes, en tenant compte de l'effectif de chaque classe (Sokolova and Lapalme, 2009). Les scores probabilistes sont calibrés par validation croisée limitée aux données d'entraînement, puisque la qualité du classement et celle des probabilités constituent deux propriétés distinctes (Niculescu-Mizil and Caruana, 2005).

1.4 Problématique et questions de recherche

La problématique retenue est la suivante:

Comment évaluer une cascade de classification sur RT-IoT2022 et conserver, pour chaque décision, les informations nécessaires à son examen par un opérateur?

Cette problématique se décompose en quatre questions:

1. comment contrôler la provenance, le schéma et l'ordre des variables utilisées pendant l'entraînement et l'inférence;

2. quelle combinaison de détecteur binaire et de classifieur de familles obtient la meilleure macro-F1 de validation selon le protocole retenu;
3. quelles erreurs sont introduites par chacun des deux étages et par la cascade complète;
4. quelles preuves supplémentaires seraient nécessaires avant toute affirmation de détection sur un réseau externe ou en temps réel.

1.5 Solution proposée et périmètre

La chaîne de préparation contrôle le fichier source, traite les doublons et fixe l'ordre des variables. L'entraînement compare les candidats sur la validation, puis mesure leurs erreurs sur le test. Une fois sélectionnés, les modèles sont enregistrés avec leurs métadonnées et servis par l'API. Chaque prédiction peut ensuite être consultée dans l'interface avec son observation source et les versions des modèles utilisés.

L'application traite des observations enregistrées et permet leur rejeu. Le laboratoire Suricata observe aussi des paquets dans un réseau isolé, par une voie indépendante de la cascade. Le transfert du modèle RT-IoT2022 vers des flux NFStream et vers un autre réseau reste à évaluer.

Le chapitre 4 présente les écrans qui permettent de consulter ces résultats et d'examiner les alertes.

1.6 Démarche de recherche

1.6.1 Mode de développement

Le travail a commencé par l'étude de RT-IoT2022 et par des essais de classification. Le développement du tableau de bord a ensuite débuté alors que les expérimentations se poursuivaient. L'historique Git montre des évolutions successives de l'API, de l'interface, de l'ingestion et de l'authentification, accompagnées de tests. La réalisation a ainsi progressé par ajustements entre ces différentes parties.

Les contrats de données ont servi de point de liaison entre les modèles, l'API et l'interface. Les tests ont ensuite accompagné les changements les plus sensibles, notamment l'ingestion, l'authentification et la persistance. L'historique Git et les rapports d'exécution permettent de retrouver les principales étapes de cette progression.

1.6.2 Phases de recherche

Les principales étapes du travail sont les suivantes :

1. audit des sources scientifiques, du fichier RT-IoT2022 et de son schéma;

2. préparation reproductible des données et définition du protocole d'évaluation;
3. comparaison des modèles, calibration, analyse des erreurs et enregistrement des modèles retenus pour l'application;
4. conception de l'API, de la persistance et de l'ingestion durable, et réalisation en parallèle de l'interface d'investigation;
5. tests unitaires, tests d'intégration, validation des artefacts et conservation des résultats et des journaux d'exécution;
6. identification des travaux non encore validés: collecte native NFStream, séparation par session ou par temps, validation externe et mesure sur le matériel cible.

La figure 1.3 synthétise le chevauchement des phases. Le développement du tableau de bord commence après les premiers travaux de recherche, puis se poursuit en parallèle des expérimentations, de l'intégration et des améliorations de l'interface. Les repères S1 à S5 représentent des périodes indicatives de travail.

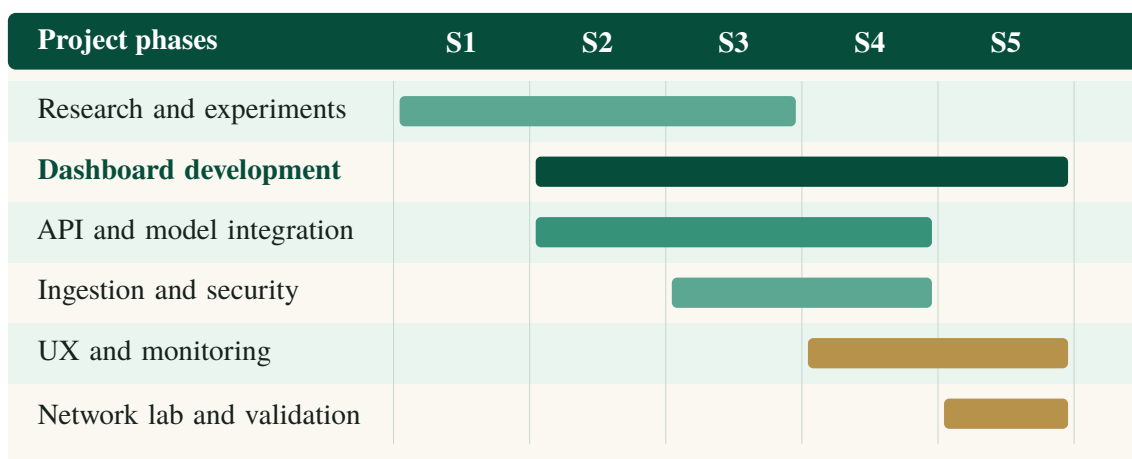


Figure 1.3 – Chronologie indicative des phases de développement

Les modèles enregistrés sont accompagnés de leurs paramètres et des résultats d'évaluation. Les journaux de tests complètent ces fichiers en décrivant les vérifications de l'application. Les résultats expérimentaux sont analysés au chapitre 5.

1.7 Conclusion

RT-IoT2022 fournit les données nécessaires à une comparaison supervisée, mais son déséquilibre et l'absence de repères temporels fiables limitent l'interprétation des scores. Le protocole doit traiter les doublons, réserver le test à l'évaluation et rendre visibles les erreurs des classes rares. L'application doit, de son côté, conserver l'origine de chaque signalement. Ces exigences orientent les besoins décrits au chapitre suivant.

CHAPITRE 2

Spécification des besoins

2.1 Introduction

L'opérateur doit pouvoir soumettre ou rejouer des observations, suivre leur traitement et examiner les alertes. Les producteurs de données ont besoin d'un contrat d'entrée stable et d'un reçu qui indique l'acceptation du lot. Les exigences ci-dessous décrivent ces parcours ainsi que le comportement attendu en cas d'erreur ou de nouvelle soumission.

2.2 Besoins fonctionnels

Tableau 2.1 – Besoins fonctionnels du prototype

ID	Besoin
BF-01	Authentifier l'opérateur et protéger les fonctions métier par un jeton à durée limitée.
BF-02	Valider puis accepter des observations de flux versionnées en mode synchrone ou par ingestion durable.
BF-03	Exécuter la cascade binaire puis, uniquement pour un flux prédit comme attaque, la classification de famille.
BF-04	Rejouer un sous-ensemble borné du jeu de données et permettre la pause, la reprise et l'arrêt.
BF-05	Persister les observations, prédictions, alertes et transitions utiles à l'audit.
BF-06	Présenter, filtrer et détailler les alertes puis enregistrer la qualification de l'analyste.
BF-07	Publier les nouvelles prédictions et alertes aux tableaux de bord connectés après validation de la transaction.
BF-08	Recevoir les événements de signature Suricata par une voie distincte et exposer l'état du capteur.
BF-09	Afficher les versions des modèles, leurs mesures d'évaluation et les instantanés de santé du modèle.

2.3 Besoins non fonctionnels

Tableau 2.2 – Besoins non fonctionnels et critères observables

Qualité	Critère observable
Intégrité	Vérifier le schéma, l'ordre des caractéristiques, les sommes de contrôle des artefacts et l'identité de chaque événement.
Fiabilité	Rendre l'ingestion idempotente, conserver les transitions et appliquer des reprises bornées avant isolement des échecs persistants.
Sécurité	Refuser par défaut les fonctions métier non authentifiées, séparer le jeton capteur et ne pas placer de secret dans l'URL WebSocket.
Réactivité	Déporter les traitements bloquants hors de la boucle asynchrone, borner les corps HTTP, les lots, la file et les connexions temps réel.
Traçabilité	Associer chaque prédiction aux versions exactes des modèles et conserver l'identité validée de l'opérateur dans l'historique.
Interopérabilité	Utiliser un contrat JSON versionné, des API HTTP et WebSocket, et distinguer explicitement les routes non compatibles.
Utilisabilité	Fournir des états de chargement et d'erreur, une navigation clavier, une adaptation mobile et des libellés qui ne transforment pas un score en certitude.

2.4 Présentation des cas d'utilisation

Les cas d'utilisation distinguent les actions de l'opérateur des échanges avec les producteurs de données. Quatre parcours sont détaillés : soumettre des flux, transmettre des événements Suricata, contrôler un rejeu et examiner une alerte. Pour chacun, la description précise les conditions d'accès, les données enregistrées et les situations d'échec.

2.5 Présentation des acteurs

L'analyste ou l'administrateur s'authentifie, contrôle le rejeu et examine les alertes. Le producteur externe soumet des observations conformes au schéma; l'agent Suricata transmet les événements du capteur avec un jeton dédié. L'attaquant intervient dans le trafic observé, sans utiliser directement les fonctions de l'application.

2.6 Diagramme global des cas d'utilisation

La figure 2.1 relie chaque objectif à l'acteur qui le déclenche. L'authentification est une précondition des parcours opérateur protégés; le capteur Suricata utilise pour sa part une identité machine distincte. L'examen d'une alerte inclut la consultation de ses preuves. L'enregistrement d'une qualification constitue une extension facultative, au point de décision de l'analyste. Les deux parcours de soumission incluent la validation des données entrantes.

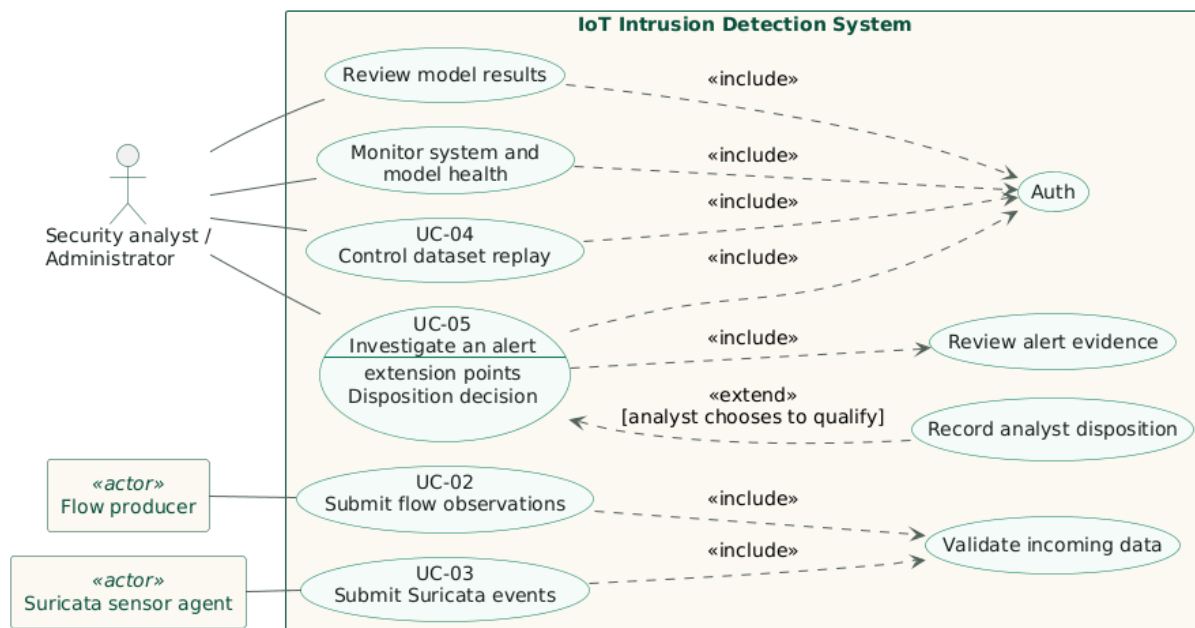


Figure 2.1 – Cas d'utilisation du système de détection

2.7 Description des cas d'utilisation

Chaque description précise les conditions d'entrée, les opérations effectuées et les données conservées. Les identifiants UC correspondent à ceux du diagramme global.

2.7.1 UC-02 — Soumettre des observations de flux

Tableau 2.3 – Description du cas UC-02 — Soumettre des observations de flux

Acteur	Producteur externe ou agent capteur utilisant le contrat d'observation autorisé.
Objectif	Soumettre d'une à mille observations canoniques à la file durable.
Préconditions	Jeton autorisé; schéma et version reconnus; identifiant d'événement fourni.
Postconditions	Chaque nouvelle observation acceptée est enregistrée dans la file avec son état initial; une soumission identique ne crée pas de second travail.
Scénario nominal	1. Le producteur envoie un lot JSON ou NDJSON. 2. L'API valide l'ensemble du lot. 3. Elle contrôle les identifiants et la capacité disponible. 4. Elle enregistre les travaux dans une transaction. 5. Elle retourne le reçu du lot.
Alternatives	Requête invalide: aucun travail stocké. Même identifiant avec contenu différent: conflit. File saturée: refus temporaire avec indication de reprise.

2.7.2 UC-03 — Transmettre des événements Suricata

Tableau 2.4 – Description du cas UC-03 — Transmettre des événements Suricata

Acteur	Producteur externe ou agent capteur.
Objectif	Transmettre une alerte de signature et l'état observé du capteur.
Préconditions	Jeton capteur valide; lot identifiant le capteur; message EVE horodaté et, pour une alerte, signature renseignée.
Postconditions	L'alerte et l'état du capteur sont persistés; un événement temps réel est produit après validation.

Scénario nominal	1. L'agent lit un événement Suricata. 2. Il le normalise. 3. L'API authentifie la machine. 4. Elle déduplique et persiste les preuves. 5. Elle retourne un reçu.
Alternatives	Jeton absent ou invalide: refus. Message non conforme: erreur de validation. Identifiant déjà accepté: traitement idempotent sans seconde alerte.

2.7.3 UC-04 — Contrôler le rejeu du jeu de données

Tableau 2.5 – Description du cas UC-04 — Contrôler le rejeu du jeu de données

Acteur	Analyste ou administrateur.
Objectif	Exécuter un rejeu borné et en contrôler la cadence.
Préconditions	Session valide; jeu préparé disponible; aucun autre rejeu actif.
Postconditions	Les lignes traitées restent persistées; l'état final indique achèvement, arrêt ou échec.
Scénario nominal	1. L'opérateur choisit scénario, décalage, limite et vitesse. 2. L'API accepte la configuration. 3. Le serveur filtre et traite les lignes. 4. L'interface suit la progression.
Alternatives	Configuration hors limites: refus. Rejeu actif: conflit. Pause: traitement suspendu. Reprise: cadence validée puis poursuite. Arrêt: abandon du reste sans annuler les lignes déjà traitées.

2.7.4 UC-05 — Examiner et qualifier une alerte

Tableau 2.6 – Description du cas UC-05 — Examiner et qualifier une alerte

Acteur	Analyste ou administrateur.
Objectif	Examiner une alerte puis, si nécessaire, enregistrer une qualification justifiée.
Préconditions	Session valide; alerte persistée.

Postconditions	La consultation ne modifie pas l’alerte; une qualification confirmée ajoute une entrée d’historique associée à l’identité authentifiée.
Scénario nominal	1. L’opérateur filtre la liste. 2. Il ouvre une alerte. 3. Il examine provenance, modèle ou signature et explication disponible. 4. Il choisit un statut. 5. Il confirme la justification.
Alternatives	Alerte absente: signalement sans modification. Explication indisponible: les autres preuves restent consultables. Justification obligatoire manquante: qualification refusée.

2.8 Conclusion

Trois comportements structurent la conception : une soumission répétée doit rester sans doublon, une alerte doit garder sa provenance et une qualification doit être associée à l’opérateur authentifié. Le chapitre suivant décrit les classes, les composants et les échanges nécessaires pour respecter ces exigences.

CHAPITRE 3

Conception du système

3.1 Introduction

La conception relie chaque observation à sa prédiction et, le cas échéant, à une alerte. Elle prévoit aussi la reprise des traitements interrompus et la publication des résultats enregistrés. Les diagrammes utilisent les noms techniques du code pour faciliter le passage du modèle à l'implémentation.

3.2 Modélisation statique

3.2.1 Modèle du domaine

La figure 3.1 relie les observations de flux, les prédictions, les alertes, les qualifications d'analyste et l'état des capteurs. Les vues suivantes détaillent les travaux d'ingestion et le suivi des modèles. Les attributs sont présentés sans indication de visibilité; le signe + désigne les opérations publiques des services.

Les classes décrivent les données enregistrées et les liens entre elles. Les services prennent en charge les traitements : accepter une observation, exécuter une prédiction ou enregistrer la qualification d'une alerte. La figure 3.4 présente leurs principales opérations.

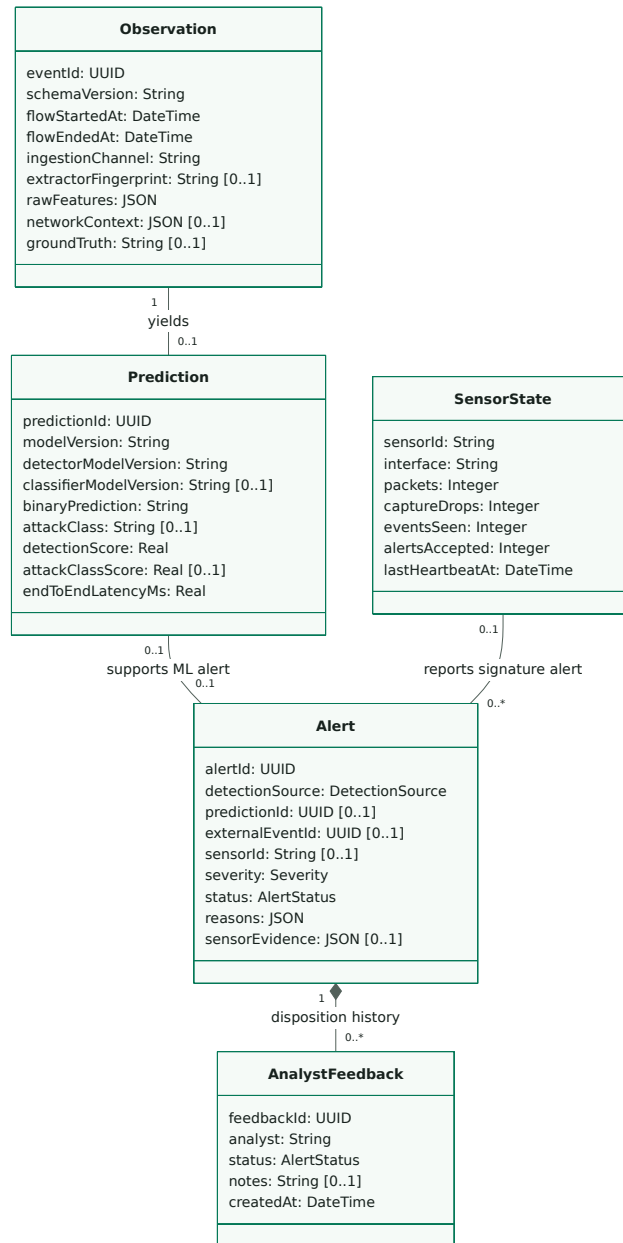


Figure 3.1 – Diagramme de classes du domaine principal

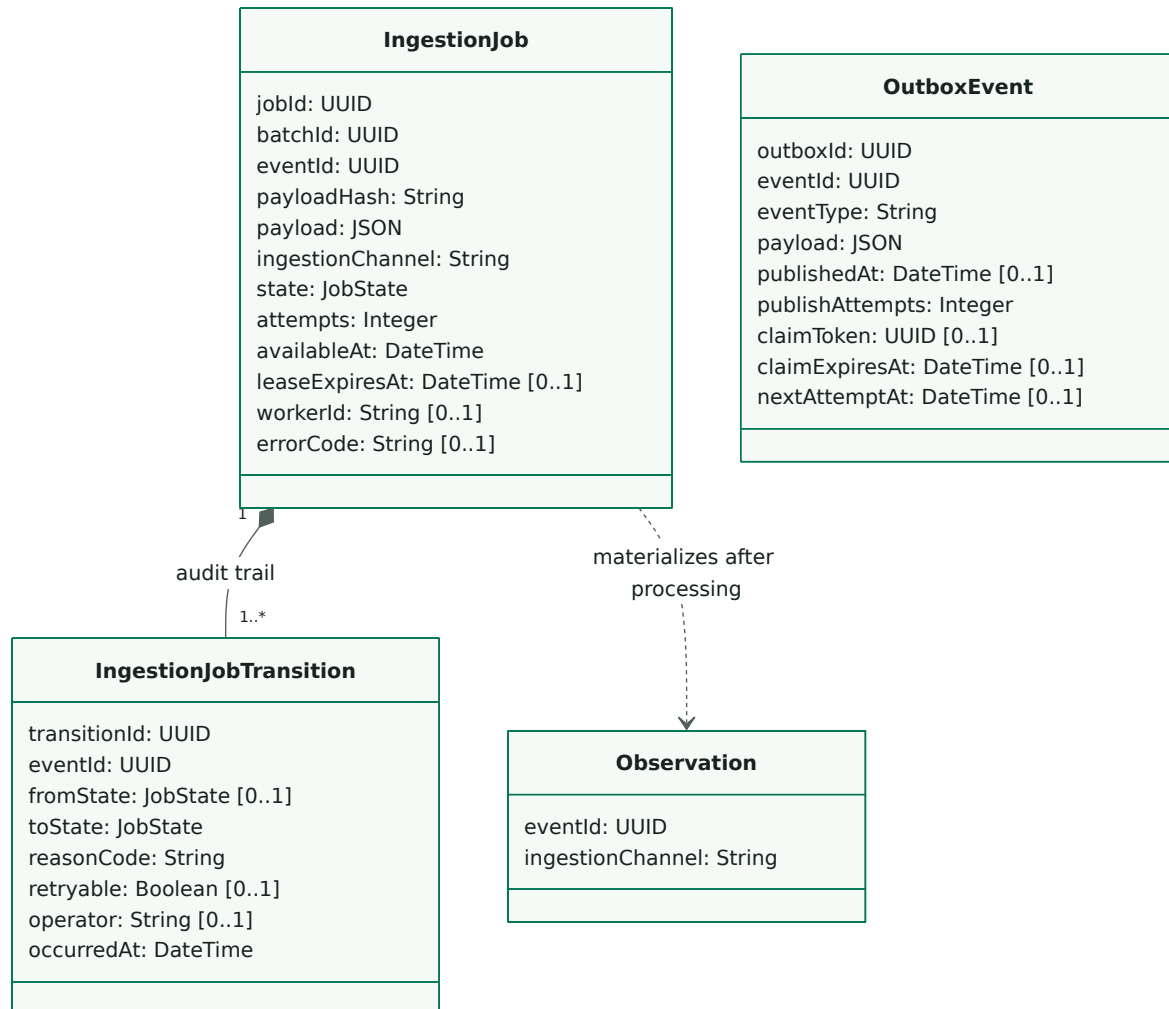


Figure 3.2 – Classes associées à l’ingestion durable

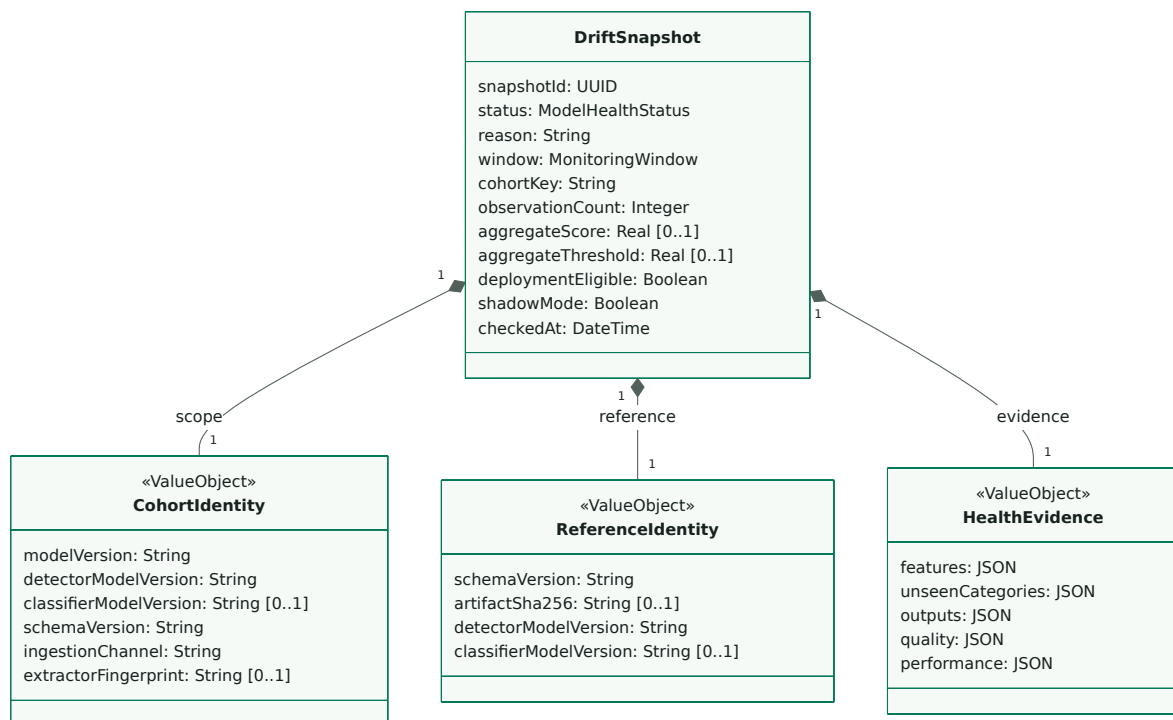


Figure 3.3 – Classes associées au suivi de l'état des modèles

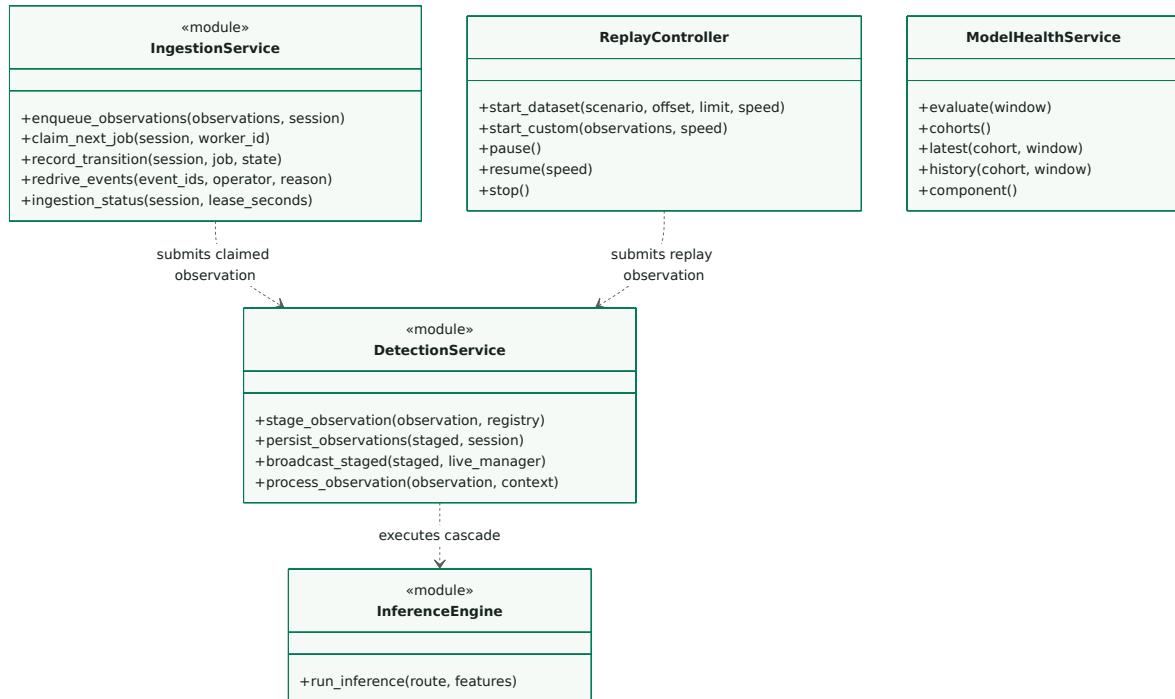


Figure 3.4 – Services et opérations principales

3.3 Architecture de l'application

L'interface communique avec l'API, qui donne accès aux modèles et aux données enregistrées (figure 3.5). La base conserve aussi les observations en attente de traitement et les messages à diffuser. Ces derniers sont placés dans une table appelée *outbox*, au sein de la même transaction que les résultats concernés. Une interruption de la diffusion n'efface donc pas les messages restant à envoyer.

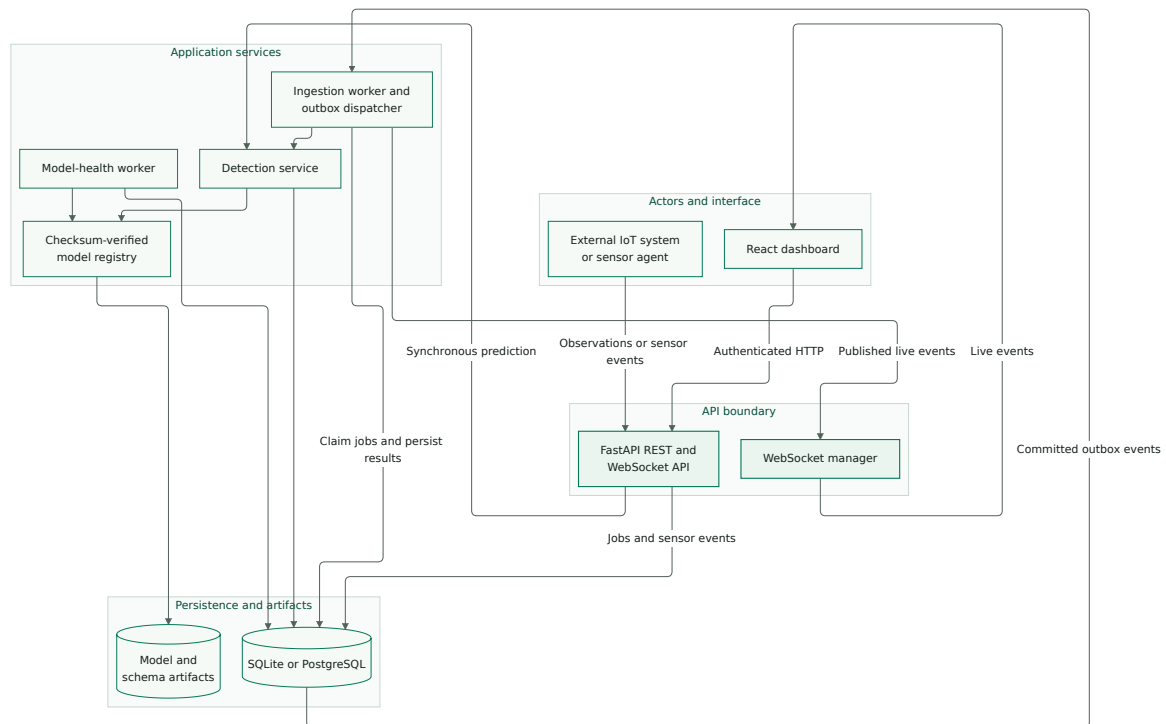


Figure 3.5 – Architecture logicielle par composants

La figure 3.6 représente uniquement le mode de démonstration Docker Compose: une passerelle web, l'API, le travailleur, la migration et PostgreSQL. Le laboratoire Suricata isolé reste un environnement de présentation distinct.

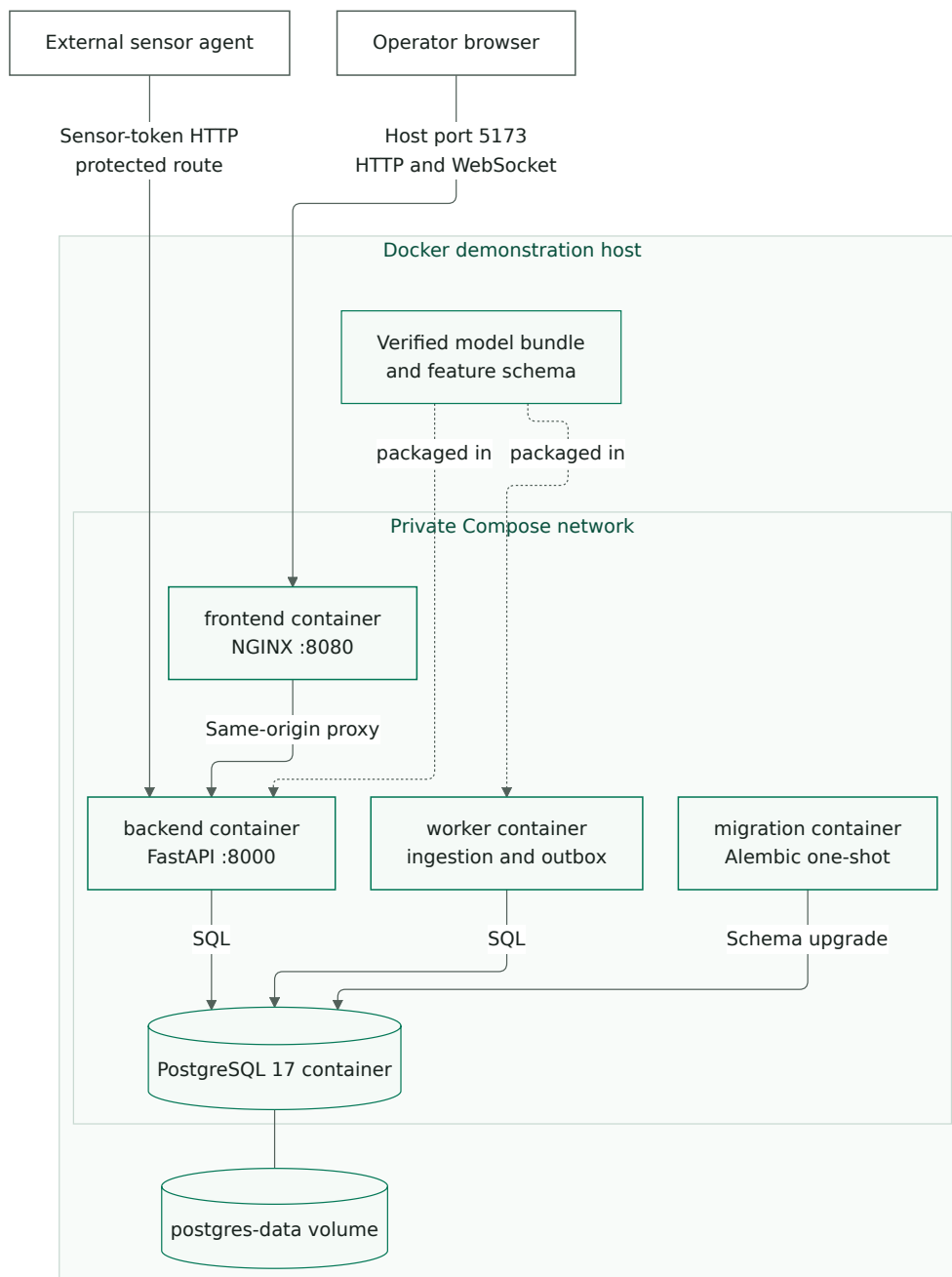


Figure 3.6 – Déploiement du mode de démonstration Docker

3.4 Modélisation dynamique

Les diagrammes dynamiques suivent les actions de l'opérateur puis les échanges entre services. La machine à états détaille le cycle de traitement d'une observation acceptée dans la file.

3.4.1 Activité de l'opérateur

Depuis la supervision, l'opérateur peut lancer un rejeu, consulter une alerte ou examiner les modèles. La figure 3.7 présente ces choix et les étapes de l'investigation.

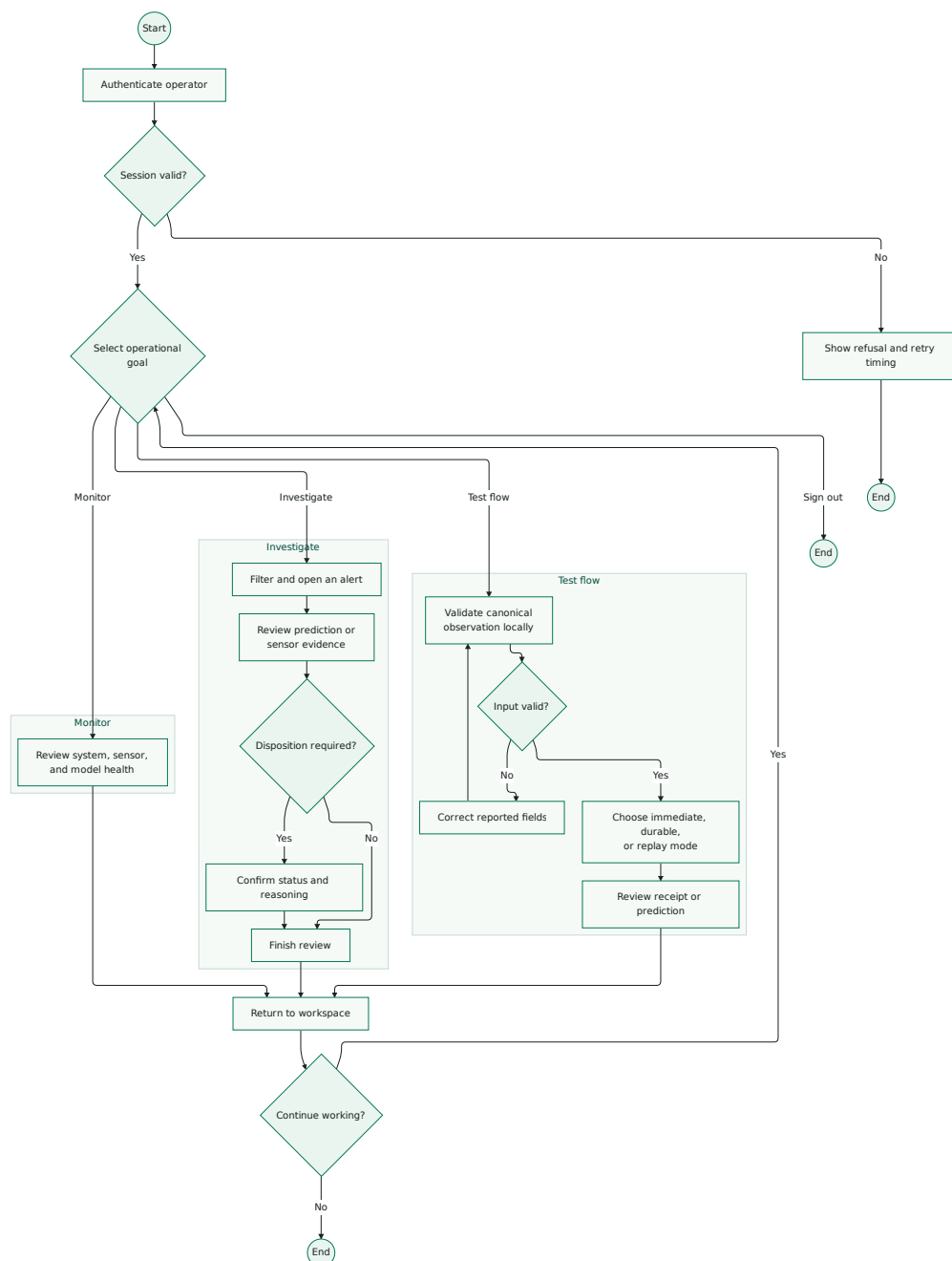


Figure 3.7 – Activité principale de l'opérateur

3.4.2 Communication pendant l'investigation

Lorsqu'un analyste ouvre une alerte, l'interface demande ses détails à l'API, qui les récupère en base. Si l'analyste confirme une qualification, une nouvelle requête enregistre sa décision. La figure 3.8 numérote ces échanges pour en montrer l'ordre.

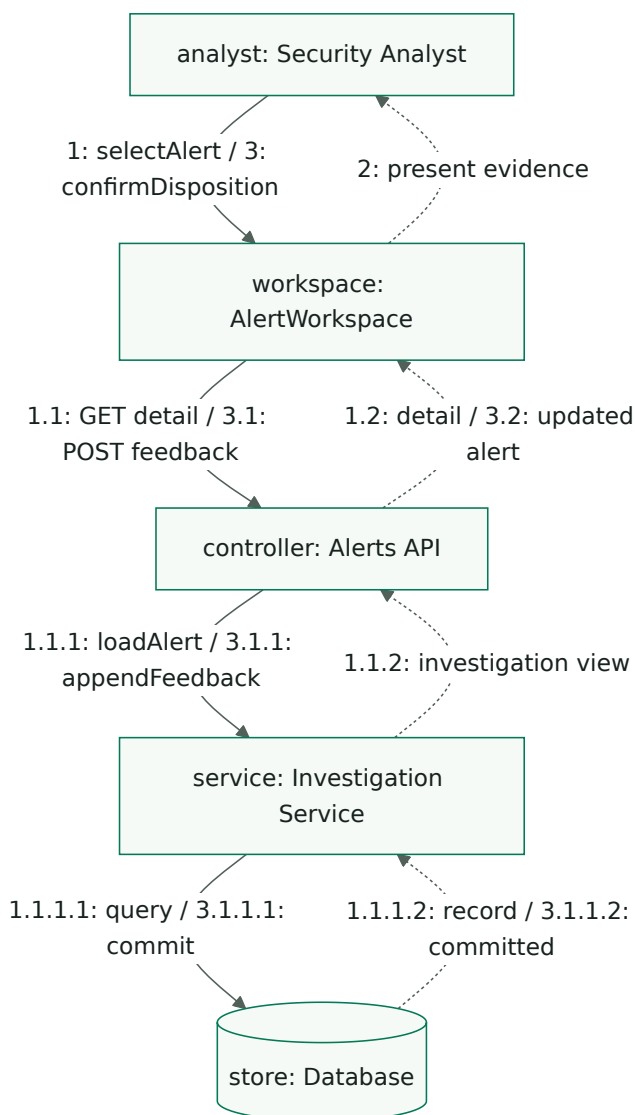


Figure 3.8 – Communication entre objets lors de l'investigation d'une alerte

3.4.3 Ingestion durable des observations

L'acceptation d'une observation et sa classification se déroulent en deux temps. L'API enregistre d'abord le travail à effectuer; un processus séparé prend ensuite en charge la prédiction. Les résultats sont enregistrés ensemble avant leur diffusion aux clients connectés. La figure 3.9 montre également les cas de rejet et de reprise après erreur. La figure 3.10 détaille les états successifs du travail en file.

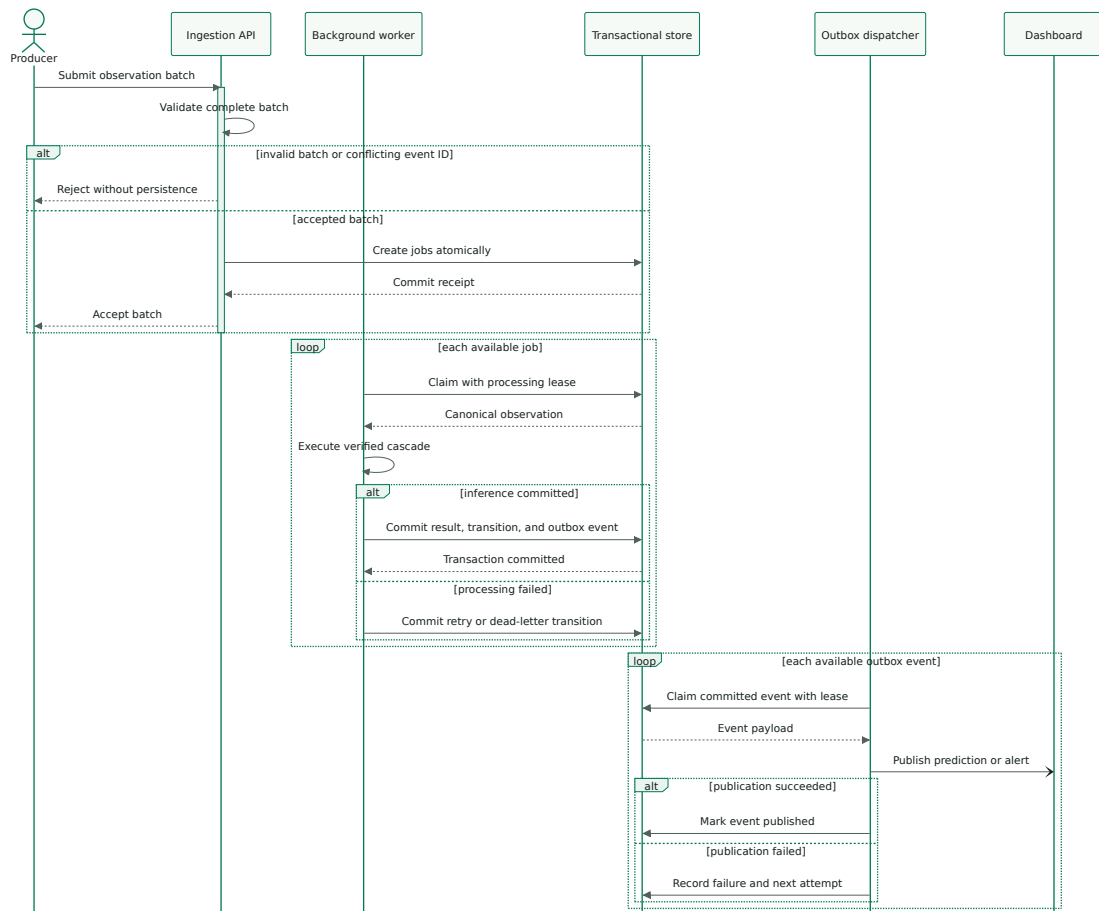


Figure 3.9 – Séquence d’ingestion durable et de diffusion des résultats

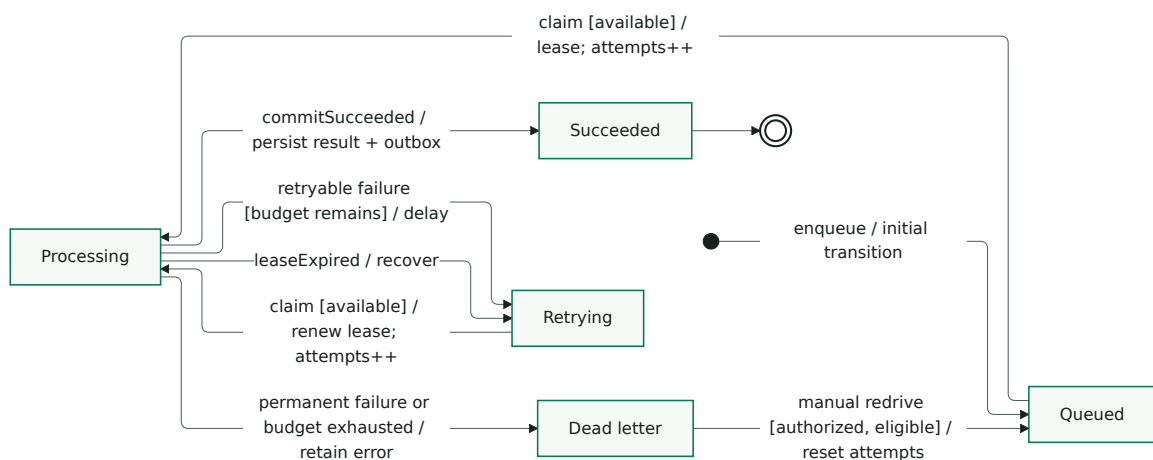


Figure 3.10 – Machine à états d’un travail d’ingestion

3.4.4 Prédiction synchrone

Dans le parcours synchrone, l'API valide le schéma, charge une cascade compatible et exécute l'inférence. Elle enregistre ensuite les résultats dans une transaction avant de répondre au client (figure 3.11).

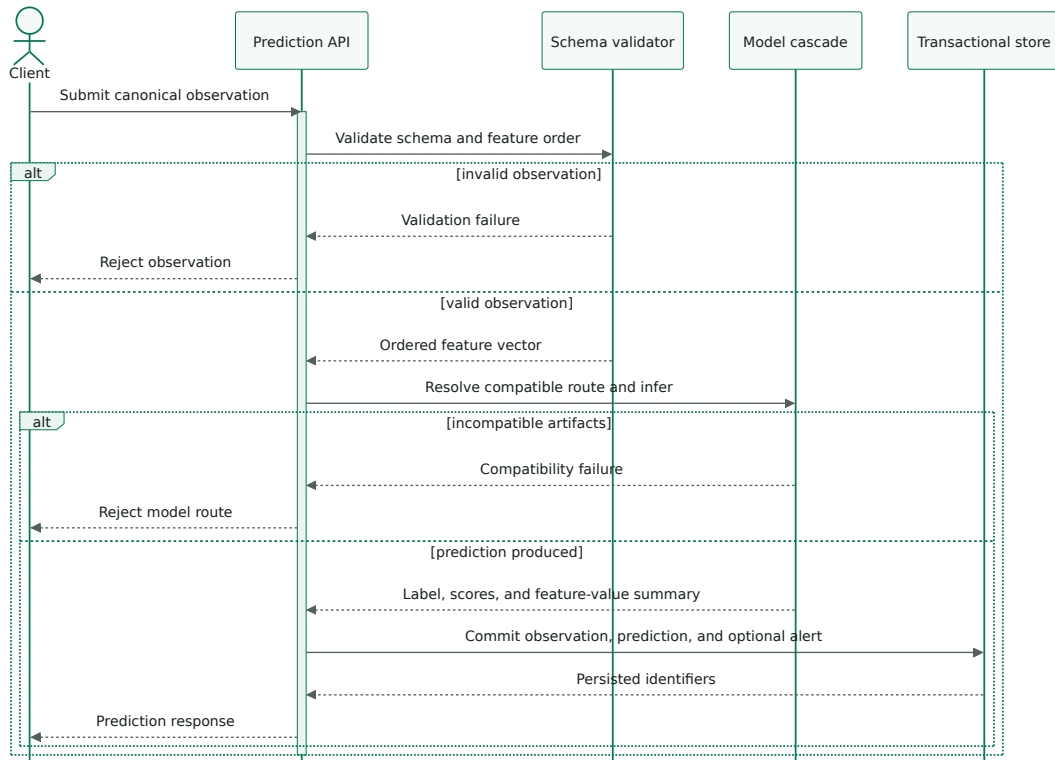


Figure 3.11 – Séquence de prédiction synchrone

3.4.5 Investigation des alertes

L'analyste consulte l'observation à l'origine de l'alerte, son modèle ou sa signature, puis les explications disponibles. Il peut ensuite enregistrer une qualification accompagnée d'une justification. Cette décision est ajoutée à l'historique avec son identité, sans remplacer le résultat initial (figure 3.12).

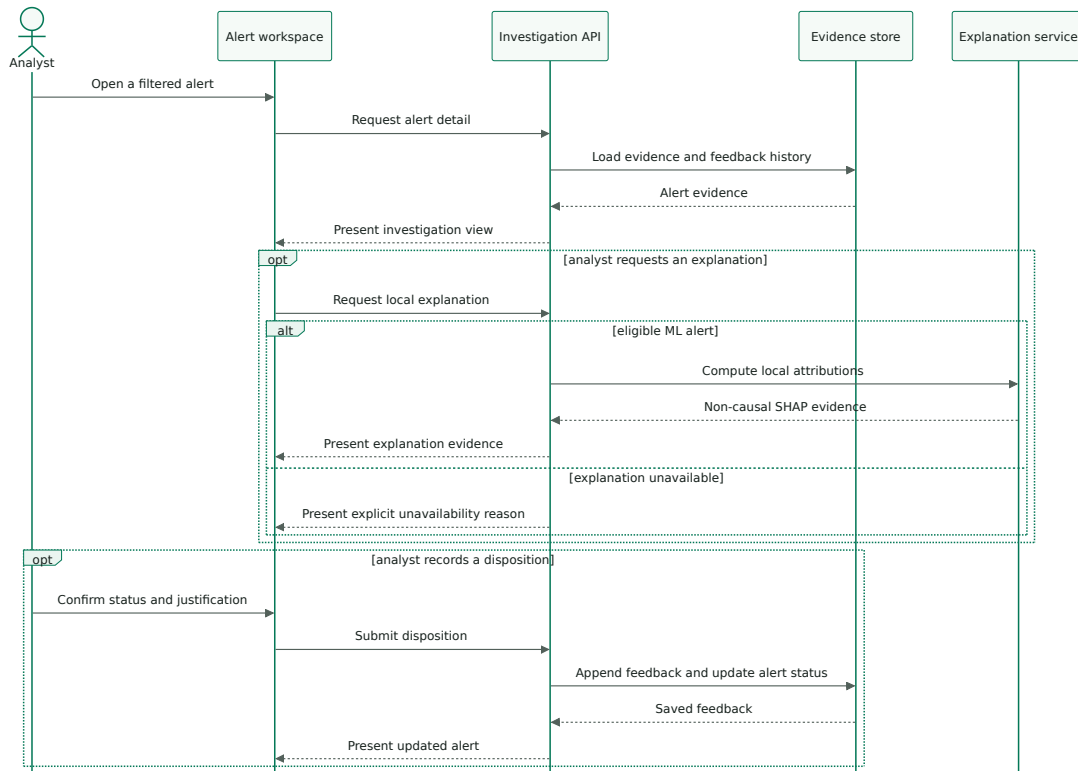


Figure 3.12 – Séquence d'investigation et de qualification d'une alerte

3.4.6 Surveillance de la santé des modèles

Le suivi des modèles compare périodiquement les observations disponibles aux données de référence du modèle. Les indicateurs calculés sont enregistrés avec leur périmètre et leur période de mesure, puis rendus consultables dans l'interface (figure 3.13). Ce suivi fonctionne en mode d'observation : il ne déclenche pas de réentraînement automatique.

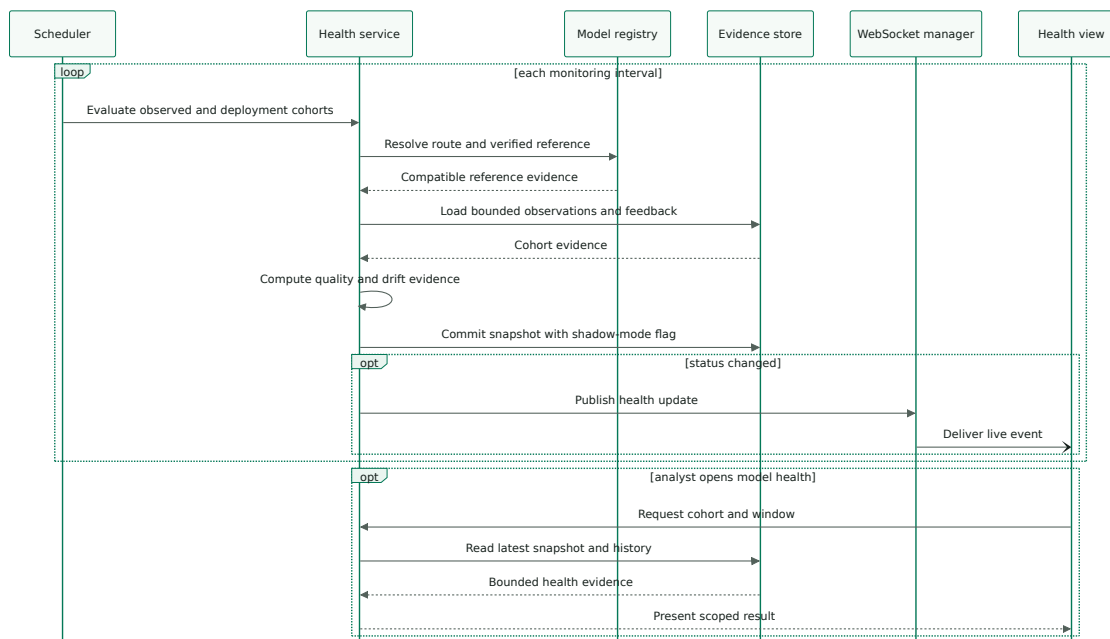


Figure 3.13 – Séquence de consultation de la santé d'un modèle

3.4.7 Authentification, rejeu et capteur

L'authentification précède l'accès aux fonctions protégées, comme l'exige BF-01. Le rejeu appelle directement le parcours de prédiction pour une suite bornée de lignes du jeu de données; il ne passe pas par la file d'ingestion durable. Les dates de début et de fin attribuées à ces observations correspondent au moment du rejeu: ce mécanisme ne reconstitue donc pas la chronologie de la capture d'origine. L'agent Suricata dispose de sa propre route d'entrée : il transmet des alertes de signature qui sont enregistrées puis publiées sans appel aux modèles de classification.

3.5 Conclusion

La base relie les observations aux décisions et conserve les transitions des travaux d'ingestion. Les services encadrent ces écritures par des transactions; l'outbox reporte la publication après leur validation. Cette organisation permet à l'interface de retrouver un résultat même si sa diffusion immédiate a été interrompue.

CHAPITRE 4

Réalisation du système

4.1 Introduction

Le prototype associe un paquet Python pour l'apprentissage, une API FastAPI et une interface React. Il s'exécute localement ou avec Docker Compose. Ce chapitre explique le rôle des outils, le traitement d'une observation et les écrans utilisés pour examiner les résultats.

4.2 Environnement de développement

4.2.1 Environnement matériel

La station sur laquelle le projet est disponible dispose d'un processeur Intel Core i5-10400F à six cœurs et douze fils d'exécution, avec environ 16 Go de mémoire vive, sous Arch Linux 64 bits. Cette configuration a été relevée le 9 septembre 2026. Le code d'apprentissage utilise des modèles Scikit-learn exécutés sur le processeur et ne requiert pas de GPU.

Ce relevé décrit le poste actuel. Les caractéristiques de la machine n'ayant pas été enregistrées avec les mesures de latence, il ne permet pas à lui seul de reconstituer les conditions matérielles de ces mesures.

4.2.2 Environnement logiciel

Neovim (nvim) a servi à l'édition du code. Le carnet Jupyter regroupe l'exploration du jeu de données et les comparaisons des modèles. Les environnements Python sont gérés avec uv, tandis que npm installe les dépendances de l'interface React et lance ses tests. Docker Compose réunit les services pour les essais d'intégration et les démonstrations.

Le relevé du 7 septembre 2026 enregistre Python 3.12.13, Node.js 26.8.0 et le noyau Linux 7.1.9-arch1-2. Le tableau 4.1 résume les autres outils mobilisés et la place qu'ils occupent dans le projet.

Tableau 4.1 – Environnement logiciel et rôle des principaux outils

Usage	Outils	Rôle
Poste	Arch Linux x86_64	Système de la station actuelle.
Édition	Neovim (nvim)	Édition du code source.
Commandes	Make et scripts Bash	Exécution des tâches et des scripts d'automatisation du dépôt.
Automatisation	Make, Git, GitHub Actions	Commandes communes, historique des modifications et validation continue.
Dépendances	Python 3.12, uv, Node.js, npm	Environnements Python et JavaScript, dépendances et fichiers de verrouillage.
Calcul et ML	NumPy, pandas, SciPy, scikit-learn, joblib, SHAP	Préparation des données, entraînement, sérialisation et explication des modèles.
Analyse	Jupyter	Exploration des données et analyse interactive des résultats.
API	FastAPI, Pydantic, Uvicorn	Validation des entrées et service des routes HTTP et WebSocket.
Données	SQLAlchemy, Alembic, SQLite, PostgreSQL 17	Accès aux données, migrations, transactions et file persistante.
Interface	TypeScript, React 19, Vite	Composants, typage, serveur de développement et compilation de l'interface.
Visualisation	D3, ECharts, Cytoscape, Lucide	Graphiques, topologie d'investigation et icônes.
Tests	pytest, Vitest, Storybook, Playwright, Axe, Ruff	Tests Python, composants, parcours navigateur, accessibilité et analyse statique.
Déploiement	Docker Engine, Docker Compose, NGINX	Construction des images, orchestration locale et passerelle web.
Réseau	Suricata, Mosquitto, NFStream	Signatures sur paquets, trafic MQTT du laboratoire et extraction PCAP hors ligne.
Sécurité et suivi	Argon2id/pwplib, PyJWT, OpenSSL, client Prometheus	Hachage, jetons, génération des secrets et exposition des métriques.
Rapport	LaTeX, latexmk, PlantUML, Graphviz, Mermaid CLI, librsvg	Composition du document et rendu vectoriel des diagrammes.

4.3 Organisation de la réalisation

Le code est réparti entre l'apprentissage, le backend et le frontend. Cette organisation permet de modifier les traitements ou les écrans séparément, tant que le format des données échangées reste compatible.

Apprentissage automatique. Le paquet `iot_ids_ml` prépare les données, entraîne les modèles et compare leurs résultats. Les modèles retenus sont enregistrés avec un manifeste qui contient les empreintes des fichiers et les références du rapport d'évaluation.

Backend. L'application FastAPI reçoit les observations, vérifie leur format et donne accès aux prédictions et aux alertes. Les travaux en attente et leur historique sont conservés en base afin de reprendre un traitement interrompu. L'outbox conserve les messages à diffuser après l'enregistrement des résultats.

Frontend. L'application React propose des espaces consacrés à la supervision, aux alertes, aux modèles, aux observations et à la topologie. Le module `api.ts` centralise les échanges avec le serveur pour que les composants se concentrent sur l'affichage et les actions de l'utilisateur.

4.4 Principaux choix d'implémentation

4.4.1 Du modèle entraîné à la prédiction servie

Pour chaque observation, le détecteur décide d'abord si le flux est normal ou suspect. Seuls les flux classés comme attaques passent au classifieur de familles. Avant d'utiliser les modèles, le registre vérifie leur schéma, l'ordre des variables et les empreintes des fichiers. Un fichier incompatible avec le manifeste actif est refusé.

Le même ordre de 83 variables est contrôlé à l'entraînement et à l'inférence. Le rejeu permet d'exercer cette chaîne avec les données enregistrées sans réentraîner le modèle. La liste `top_features` renvoyée avec une prédiction classe cinq valeurs d'entrée selon leur magnitude absolue; elle ne mesure pas leur influence sur le modèle. Pour les alertes ML compatibles avec les versions actives, un point d'accès distinct calcule à la demande des contributions SHAP (Lundberg and Lee, 2017). Dans le cas des modèles calibrés, ces contributions expliquent la marge de l'estimateur arborescent ajusté du premier pli de calibration, et non la probabilité calibrée. Elles aident à lire une prédiction, sans établir les causes d'une attaque.

4.4.2 Ingestion durable et traitement des erreurs

L'ingestion HTTP valide un lot complet avant toute insertion. L'identifiant d'événement et l'empreinte du contenu assurent l'idempotence. Le travailleur réserve ensuite un travail, exécute la

cascade et valide dans une transaction l'observation, la prédiction, l'alerte éventuelle, la transition et les messages d'outbox. La diffusion temps réel ne commence qu'après cette validation.

Une nouvelle soumission identique n'ajoute pas une seconde observation ; un contenu différent sous le même identifiant est refusé. Après une erreur temporaire, le travail est repris avec un délai et un nombre d'essais bornés. Les échecs persistants passent en état `dead_letter`. Une relance manuelle authentifiée est disponible pour les travaux encore compatibles avec la route de modèle active; le motif et la nouvelle transition sont conservés.

4.4.3 Interface, contrôle d'accès et provenance

Les indicateurs de l'interface ne portent pas tous sur les mêmes données. Certains résument l'historique enregistré en base, d'autres concernent les alertes chargées ou les événements reçus depuis l'ouverture du navigateur. Leur périmètre est indiqué pour que ces valeurs puissent être interprétées correctement.

La session opérateur utilise un jeton à durée limitée ; l'interface traite son expiration et les refus de l'API. L'identité associée à une qualification provient du jeton validé, et non d'un nom libre fourni par le navigateur. Les vues présentent les états de chargement, d'erreur et de reconnexion pour éviter qu'une donnée ancienne soit confondue avec un état actualisé.

Dans le laboratoire, Suricata observe le trafic MQTT et HTTP du réseau Docker isolé. L'agent transmet les événements avec une identité machine distincte. L'interface affiche leur origine de signature ; ils ne traversent pas la cascade RT-IoT2022. NFStream reste une voie d'extraction PCAP hors ligne distincte.

4.4.4 Exécution reproductible

Pour retrouver rapidement un environnement cohérent, les commandes courantes ont été regroupées dans le Makefile. `make setup` installe les dépendances et `make verify-model` contrôle que les modèles chargés correspondent bien à leur manifeste. La commande `make demo` crée une base SQLite temporaire, puis lance par l'API le jeu d'un nombre configurable d'observations normales et d'attaque en proportions égales. `make demo-sensor` démarre un environnement complet distinct, comprenant le laboratoire Suricata. Les scripts appliquent les migrations et vérifient la disponibilité des services au démarrage.

4.5 Principales interfaces graphiques

4.5.1 Supervision opérationnelle

L'écran de supervision permet de vérifier la disponibilité des services, de suivre un rejeu et de repérer les travaux d'ingestion en attente ou en échec. Les indicateurs distinguent les résultats enregistrés des événements reçus pendant la session (figure 4.1).

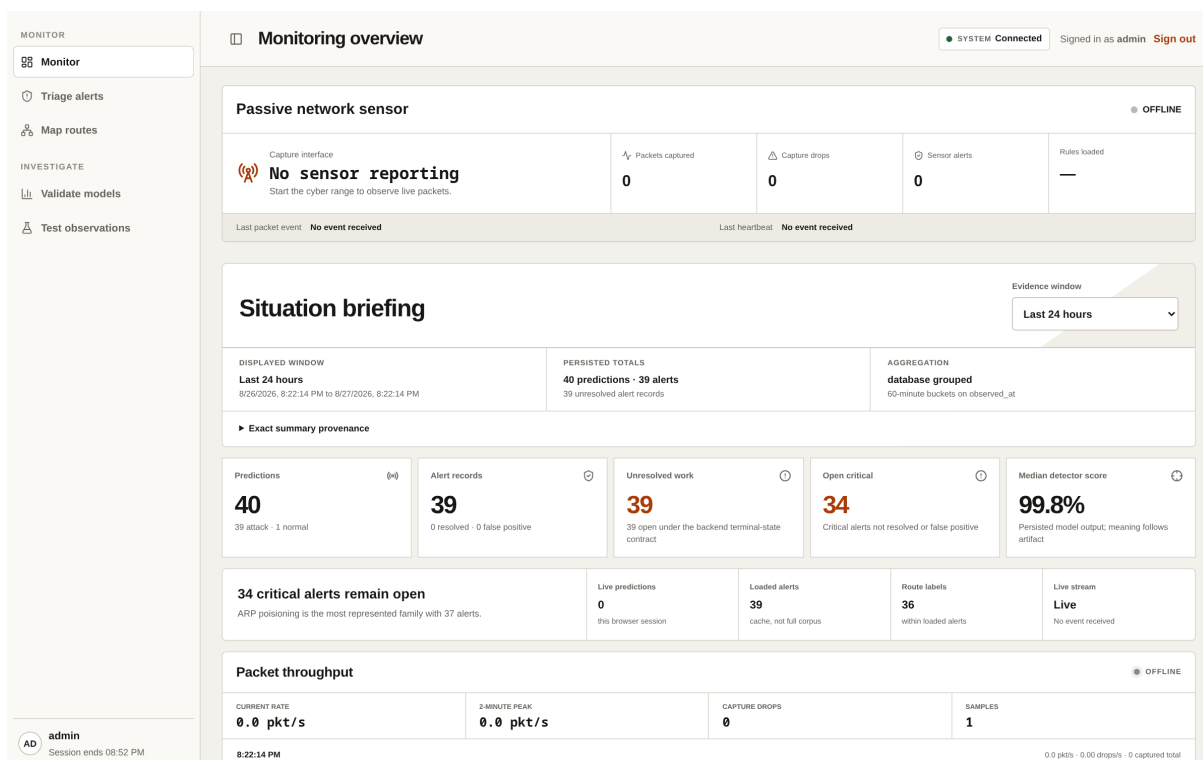


Figure 4.1 – Espace principal de supervision du prototype

4.5.2 Investigation des alertes

L'opérateur filtre la liste des alertes, puis ouvre celle qu'il souhaite examiner. Le détail indique si le signalement provient d'un modèle ou de Suricata et présente les scores, les versions et les qualifications disponibles (figure 4.2).

Alert investigation SYSTEM Connected Signed in as admin [Sign out](#)

Security alerts 39
matching alerts · newest first

Quick views: **All alerts** Needs review Critical In progress

Search: Time: Detection: Severity: Status:

SEVERITY	DETECTION	ROUTE	PROTOCOL	EVIDENCE	STATUS	TIME
CRITICAL	ARP_poisoning 7beb756b-8792-4df1-a66d-8bee443ce55e	port 54835 to port 53	udp	100.0% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 25b99088-e833-4773-9b24-f79225459540	port 38680 to port 53	udp	100.0% model	New	07:22:13 PM
CRITICAL	ARP_poisoning b88c40ba-2715-4191-846d-c39a496069ad	port 46956 to port 443	tcp	100.0% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 65006096-278c-4f28-9b83-89619ed15e19	port 46962 to port 443	tcp	99.9% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 13fd6d85-dd61-4cab-bf48-e27a623ec25b	port 46958 to port 443	tcp	99.9% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 58131d04-3453-4c40-8d68-9fcfc3d145cd	port 46960 to port 443	tcp	100.0% model	New	07:22:13 PM
CRITICAL	ARP_poisoning b8daf703-24f0-45f9-0022-a2d5a9254b89	port 5353 to port 5353	udp	98.6% model	New	07:22:13 PM
CRITICAL	ARP_poisoning 0d552830-4bf4-467e-89b3-264df82ad385	port 5353 to port 5353	udp	98.6% model	New	07:22:13 PM

AD admin
Session ends 08:52 PM

Figure 4.2 – Espace d’investigation des alertes

4.5.3 Analyse des modèles

L'écran d'analyse rassemble les résultats d'évaluation des modèles actifs. Les partitions et les graines y sont précisées pour permettre à l'opérateur de retrouver les conditions du calcul des scores (figure 4.3).

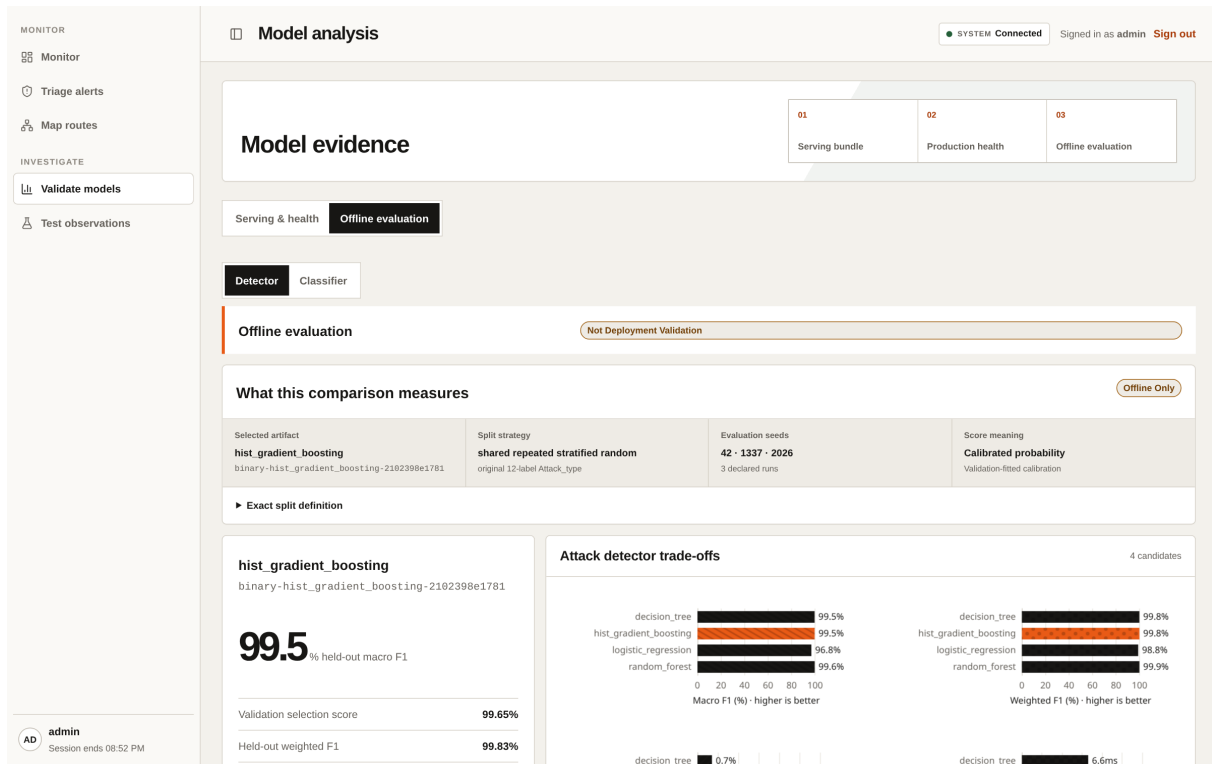


Figure 4.3 – Espace d'analyse et de traçabilité des modèles

4.5.4 Laboratoire d'observations

Le laboratoire d'observations permet de préparer une observation ou un petit lot, puis d'en vérifier le format avant l'envoi. Les erreurs détectées sont affichées pour faciliter la correction. L'API effectue ensuite ses propres contrôles avant d'accepter les données (figure 4.4).

MONITOR

- Monitor
- Triage alerts
- Map routes

INVESTIGATE

- Validate models
- Test observations

Observation lab

SYSTEM Connected Signed in as admin Sign out

Observation lab

1 Choose observations

RT-IoT2022 CSV file
CSV only - maximum 10 MB - one file

Choose File No file chosen

Or drop one CSV here

Load verified normal example Load verified attack example

RT-IoT2022 extracted SHA-256
956956c09c1764584fa88acd9f6876475626bcdcd6a6b1f8c492c2e9a2b89ea. Source lines 2 and 12509; only the dataset index outside the serving schema is removed.

SELECTED FILE
rt-iot2022-line-12509-attack.csv

ROWS	FEATURE COLUMNS	REFERENCE LABELS
1	83	1

2 Review contract and values

Header contract
83 feature names in canonical order

Cell values
81 numeric and 2 categorical fields validated per row

Sample preview
First 1 rows - selected identifying and flow fields

Row	id.orig_p	id.resp_p	proto	service	flow_duration	fwd_pkts_tot	bwd_pkts_tot	Attack_type
1	5353	5353	udp	dns	0	1	0	ARP_posic

The complete 83-feature vector is submitted; this focused preview supports recognition without presenting an 84-column page.

3 Choose processing path

☒ **Analyze immediately**
Persists predictions, creates alerts for attack verdicts, and returns every result in this workspace.
Best for: Controlled validation and row-by-row comparison
LIMIT 10,000 ROWS

☐ **Queue reliably**
Persists jobs for worker processing, retry, dead-letter recovery, and outbox delivery evidence.
Best for: Recoverable asynchronous ingestion
LIMIT 1,000 ROWS

☐ **Replay as live traffic**
Starts a custom replay through the detector, persistence layer, and live event stream.
Best for: End-to-end exercises using uploaded observations
LIMIT 100,000 ROWS

admin
Session ends 08:52 PM

Figure 4.4 – Laboratoire de validation et de soumission d'observations

4.6 Validation technique

La validation combine des tests des traitements individuels et des essais de l'application complète. Les tests Python couvrent la préparation des données, les modèles, l'API, la persistance et l'ingestion. Vitest contrôle les adaptateurs et les composants React, tandis que Playwright reproduit dans un navigateur les actions principales d'un opérateur. Les commandes `make test`, `make lint`, `make build` et `make project-preflight` rassemblent ces contrôles avant une démonstration.

Les tests Storybook exercent les interactions des composants isolés. D'autres contrôles comparent les chiffres du support de maintenance au rapport d'évaluation. Le fichier `machine-learning/reports/generated/presentation-evidence.json` enregistre une répétition terminée le 7 septembre 2026 : les trois groupes de vérifications (projet, déploiement Docker temporaire et laboratoire Suricata) portent le statut pass. Le journal du projet rapporte dix parcours navigateur réussis; celui du laboratoire vérifie trois scénarios de signature. Ces résultats décrivent cette exécution locale. Ils ne mesurent pas la qualité de détection sur un réseau externe.

4.7 Difficultés rencontrées et solutions apportées

Trois difficultés ont particulièrement marqué le travail : comprendre les données, fixer une limite aux expérimentations et rendre le tableau de bord facile à lire.

4.7.1 Comprendre les caractéristiques du jeu de données

Au début du projet, la principale difficulté était de comprendre ce que représentaient les variables de RT-IoT2022. Il était possible de lancer un entraînement sans pour autant savoir interpréter une durée de flux, un compteur de paquets ou une mesure calculée dans un sens de communication. L'étude de la description du jeu de données et l'exploration dans Jupyter ont aidé à relier ces caractéristiques au trafic observé. L'examen des distributions, des valeurs égales à zéro et des doublons a également permis de mieux comprendre les données avant de comparer les modèles.

4.7.2 Savoir quand passer des essais à l'application

La comparaison des modèles pouvait facilement se prolonger : chaque résultat donnait envie d'essayer un autre réglage. Il fallait pourtant avancer sur l'application. Le protocole présenté au chapitre 5 a donné un cadre à cette étape : comparer quatre familles sur les mêmes partitions, retenir les modèles selon leur macro-F1 de validation et réserver le test à l'évaluation finale. Cette règle a permis de fixer une version de référence pour l'intégration, tout en conservant les autres essais pour l'analyse.

4.7.3 Rendre le tableau de bord compréhensible

Les premières versions du tableau de bord affichaient beaucoup de chiffres, mais il n'était pas évident de savoir lesquels regarder en premier. La mise en page a donc été reprise plusieurs fois. Les vues actuelles séparent la supervision, l'investigation des alertes et l'analyse des modèles. Cette organisation permet de commencer par une vue d'ensemble, puis de consulter les détails utiles à une question précise. Le travail sur les libellés et sur la distinction entre historique enregistré et activité de la session a également aidé à clarifier la lecture des indicateurs.

4.8 Conclusion

Le prototype permet de soumettre une observation, d'enregistrer sa prédiction et d'examiner l'alerte correspondante. Les journaux de la répétition documentent le fonctionnement des parcours applicatifs et du laboratoire. Le chapitre suivant examine une autre question : les erreurs commises par les modèles sur les données de test.

CHAPITRE 5

Résultats expérimentaux

5.1 Données et fichiers de référence

Les résultats de ce chapitre proviennent de `models/production/evaluation-report.json`. Le manifeste de production associe ce rapport aux deux modèles sélectionnés par leurs empreintes SHA-256. Le CSV RT-IoT2022 contient 123 117 lignes et 83 variables explicatives; son empreinte commence par 956956c09c17.

Le rapport d'entraînement référence le commit 531fe6ae0443. Le champ `code_worktree_dirty` vaut `true` : des modifications non committées étaient présentes lors de sa génération. Les empreintes identifient les fichiers conservés, mais le commit seul ne permet pas de retrouver exactement le code qui les a produits.

5.2 Qualité, nettoyage et déséquilibre

L'audit ne relève aucune valeur manquante ou infinie. Il trouve six groupes de vecteurs identiques portant des étiquettes contradictoires, soit 128 lignes. Le nettoyage supprime toutes ces lignes, puis retire 5 080 copies supplémentaires de vecteurs ayant la même étiquette. Il reste donc 117 909 observations. Le nombre de 5 202 indiqué au tableau 1.1 compte les lignes excédentaires à caractéristiques identiques avant ces deux opérations; il ne désigne pas 5 202 groupes de doublons.

La distribution source est très déséquilibrée: `DOS_SYN_Hping` représente 94 659 lignes, alors que `NMAP_FIN_SCAN` n'en contient que 28 et `Metasploit_Brute_Force_SSH` 37. Pour cette raison, l'exactitude globale n'est pas utilisée seule; la macro-F1, le rappel par classe, le support et le taux de faux positifs sont publiés.

L'analyse descriptive montre en outre des variables très asymétriques et de nombreux zéros. Les graphiques emploient donc une transformation `log1p` uniquement pour la visualisation. Les associations avec la cible restent descriptives: elles peuvent révéler des artefacts de scénario et ne sont pas interprétées comme des causes d'attaque.

5.3 Protocole expérimental

Pour chaque graine (42, 1337 et 2026), les observations nettoyées sont réparties par tirage stratifié selon l'étiquette d'origine : 70 745 lignes pour l'entraînement, 23 582 pour la validation et 23 582 pour le test, soit environ 60 % / 20 % / 20 %. Tous les candidats utilisent les mêmes partitions pour une graine donnée. Le second étage utilise uniquement les attaques de ces partitions, tandis que la cascade est évaluée sur l'ensemble du test.

Le prétraitement et la calibration sigmoïde à trois plis sont ajustés sur l'entraînement. Chaque famille de modèles est classée selon sa macro-F1 moyenne de validation sur les trois graines. En cas d'égalité, la procédure utilise successivement le taux de faux positifs, la taille sérialisée et l'ordre alphabétique du nom du modèle. Les scores de test sont calculés pour les candidats, mais n'entrent pas dans cette règle de sélection.

La F1 combine la précision (la part des prédictions correctes pour une classe) et le rappel par leur moyenne harmonique. La macro-F1 est la moyenne non pondérée des F1 par classe. Elle donne le même poids à chaque classe, quel que soit son effectif. Le rappel mesure la part des observations d'une classe correctement reconnues; le support est le nombre d'observations de cette classe. Pour le détecteur, le taux de faux positifs (FPR) est la proportion de flux normaux classés comme attaques. Pour les familles, le FPR macro est la moyenne des taux calculés en opposant chaque famille à toutes les autres. Ces deux FPR n'ont donc pas le même dénominateur.

Le tableau de données ne permet pas une partition fiable par session, appareil ou date. Les trois tirages mesurent la sensibilité au partage de ce même corpus; ils ne constituent pas trois collectes indépendantes.

5.4 Comparaison des classifieurs

Tableau 5.1 – Comparaison du détecteur binaire sur les trois graines

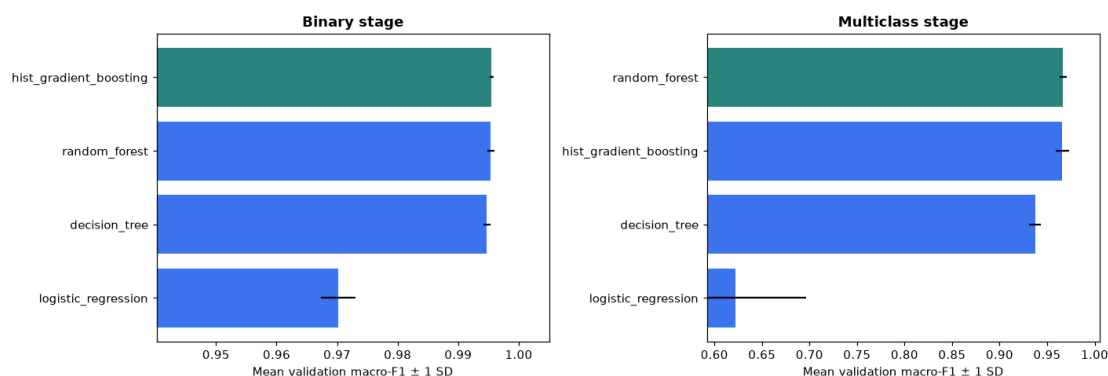
Modèle	Macro-F1 val.	σ	FPR val.	Macro-F1 test 42
Régression logistique	0,9702	0,0028	5,5371%	0,9684
Arbre de décision	0,9947	0,0006	0,7494%	0,9945
Forêt aléatoire	0,9954	0,0006	0,9992%	0,9949
Gradient boosting	0,9955	0,0003	0,8604%	0,9961

Le gradient boosting est retenu pour la détection et la forêt aléatoire pour les familles. Les écarts de validation entre les meilleurs candidats sont faibles; la règle de sélection déclarée, et non le score de test, détermine les modèles retenus. La figure 5.1 montre la restitution graphique produite par le carnet d'analyse. Dans les deux tableaux, les colonnes de validation sont des moyennes sur les trois graines et σ est l'écart-type de leurs macro-F1, calculé avec un diviseur

Tableau 5.2 – Comparaison du classifieur des familles d’attaque

Modèle	Macro-F1 val.	σ	FPR macro val.	Macro-F1 test 42
Régression logistique	0,6222	0,0745	0,2357%	0,6308
Arbre de décision	0,9375	0,0060	0,0124%	0,9602
Gradient boosting	0,9659	0,0070	0,0072%	0,9619
Forêt aléatoire	0,9668	0,0039	0,0104%	0,9642

de trois. La dernière colonne rapporte seulement le test de la graine 42. Les faibles écarts de validation ne suffisent pas à établir une supériorité statistique.

**Figure 5.1** – Comparaison des candidats dans le notebook d’analyse

5.5 Expérience sur le déséquilibre

Une expérience diagnostique conserve l’architecture HistGradientBoosting et les mêmes partitions de la graine 42. Elle compare l’absence de correction, la pondération des classes et un sous-échantillonnage aléatoire limité à l’entraînement. Cette expérience ne participe pas à la promotion.

Tableau 5.3 – Comparaison contrôlée des traitements du déséquilibre

Étage	Traitement	Lignes ajustées	Macro-F1 val.	Macro-F1 test
Binaire	Sans correction	70 745	0,9955	0,9969
Binaire	Pondération	70 745	0,9946	0,9961
Binaire	Sous-échantillonnage	14 412	0,9891	0,9918
Familles	Sans correction	63 539	0,9528	0,9668
Familles	Pondération	63 539	0,9368	0,9551
Familles	Sous-échantillonnage	153	0,7150	0,7150

La pondération augmente le rappel du trafic normal, classe binaire minoritaire, de 0,9946 à 0,9975. Dans ce diagnostic, elle réduit donc les fausses alertes, mais ne maximise pas la macro-F1. Le sous-échantillonnage ramène chaque classe à l’effectif de la plus rare : il ne conserve que 153

lignes pour l'apprentissage des familles, dont la macro-F1 de test tombe à 0,7150. Ce résultat concerne ce réglage et ce partage; il ne permet pas d'écarter toute forme de sous-échantillonnage.

5.6 Erreurs des modèles retenus et de la cascade

La matrice binaire du test 42 contient 21 159 attaques correctement détectées, 20 attaques manquées, 14 faux positifs et 2 389 flux normaux correctement reconnus. Le taux de faux positifs vaut donc $14/(14 + 2\,389)$, soit 0,5826 %.

Tableau 5.4 – Classes les plus fragiles dans la cascade, graine 42

Classe	Support	Rappel	F1
NMAP_FIN_SCAN	6	0,8333	0,7692
Metasploit_Brute_Force_SSH	7	0,7143	0,8333
DDOS_Slowloris	106	0,9528	0,9573
NMAP_TCP_scan	200	0,9950	0,9975

La cascade obtient une macro-F1 de 0,9520 pour la graine 42 et de 0,9686 en moyenne sur les trois graines. Les trois étiquettes normales sont regroupées en une classe, ce qui donne dix classes pour cette mesure. Une erreur sur les six flux NMAP_FIN_SCAN fait varier le rappel de 16,7 points de pourcentage; deux des sept attaques SSH sont mal classées. Ces petits effectifs expliquent pourquoi les résultats par classe sont nécessaires pour lire le score global.

5.7 Calibration, seuil et coût opérationnel

Les deux modèles retenus utilisent une calibration sigmoïde. Sur le test de la graine 42, le score de Brier vaut 0,0020 pour le détecteur et 0,0015 pour le classifieur de familles. L'implémentation calcule la moyenne, sur les lignes, de la somme des erreurs quadratiques de probabilité sur toutes les classes. Cette convention inclut les deux classes dans le cas binaire. L'erreur de calibration attendue (ECE), calculée sur la confiance maximale avec dix intervalles de même largeur, vaut respectivement 0,0008 et 0,0010. Ces mesures décrivent les probabilités sur ce test; la calibration devra être réévaluée en cas de changement de trafic (scikit-learn developers, s.d.).

Le seuil de service du détecteur reste fixé à 0,5. Le rapport conserve une courbe calculée sur la validation pour examiner le compromis entre fausses alertes et attaques manquées. Cette analyse n'a pas modifié le seuil.

La mesure de latence appelle `predict` séparément sur chacune des 1 000 premières observations de validation, après un appel de préparation. La moyenne des trois p95 vaut 19,27 ms pour le détecteur et 31,79 ms pour le classifieur de familles. Il s'agit de la moyenne de trois percentiles, et non du p95 de toutes les durées réunies. Les fichiers des modèles de la graine 42

occupent respectivement 418 481 et 1 325 252 octets. Ces durées portent sur l'inférence seule; elles n'incluent ni le transport HTTP ni l'écriture en base.

5.8 Menaces à la validité

L'évaluation reste limitée à RT-IoT2022. Le nettoyage élimine les doublons exacts, mais ne garantit pas l'indépendance entre des flux d'un même scénario. Les classes rares ont peu de représentants dans le test et les partitions aléatoires peuvent partager des caractéristiques propres à la collecte. Une évaluation par session, par appareil ou sur un autre réseau reste à mener.

La reproduction de l'expérience rencontre aussi deux limites : le code comprenait des modifications non committées et le rapport ne décrit pas précisément le matériel de mesure. Enfin, aucun nombre de fausses alertes par heure ne peut être déduit de ce test sans horodatages fiables et sans cadence de trafic représentative du réseau cible.

5.9 Conclusion

La règle de sélection retient le gradient boosting par histogrammes au premier étage et la forêt aléatoire au second. Leurs erreurs montrent que le score global masque des difficultés sur les classes les moins représentées. Le résultat fournit un point de comparaison pour une prochaine collecte, à condition d'enregistrer les paramètres d'extraction et de séparer les sessions entre entraînement et test.

Conclusion générale

Le projet relie une expérience de classification sur RT-IoT2022 à une application d'investigation. La préparation contrôle le fichier source et retire les conflits d'étiquettes ainsi que les doublons. La comparaison de quatre familles de modèles retient un gradient boosting par histogrammes pour la détection binaire et une forêt aléatoire pour les familles d'attaque. L'API enregistre leurs décisions, et l'interface permet de retrouver les observations, les versions des modèles et les qualifications de l'analyste.

La cascade atteint une macro-F1 de 0,9520 sur le test de la graine 42 et une moyenne de 0,9686 sur les trois graines. Les résultats sont moins stables pour les classes rares, dont certaines ne comptent que six ou sept observations de test. Les partitions aléatoires d'un même corpus ne renseignent pas encore sur le comportement du modèle dans un autre environnement.

Sur le plan applicatif, la file persistante, les reprises et la publication après transaction permettent de suivre le traitement d'une observation jusqu'à sa consultation. La répétition locale documente ces parcours et les scénarios de signature Suricata. La traçabilité reste toutefois incomplète pour une reproduction exacte de l'entraînement : le rapport signale des modifications non commitées, et le matériel des mesures de latence n'est pas détaillé.

La prochaine expérience devrait partir d'une collecte NFStream étiquetée, avec les paramètres d'extraction, les sessions et les horodatages conservés. Elle permettrait de séparer les sessions d'entraînement et de test, puis de mesurer les attaques manquées et les fausses alertes sur un trafic représentatif. L'archivage du code exact et des caractéristiques de l'hôte compléterait les fichiers déjà conservés. Ces éléments permettraient de juger le transfert du modèle au-delà de RT-IoT2022.

Bibliographie et webographie

- Zied Aouini and Adrian Pekar. NFStream: A flexible network data analysis framework. *Computer Networks*, 204:108719, February 2022. ISSN 13891286. doi: 10.1016/j.comnet.2021.108719. URL <https://linkinghub.elsevier.com/retrieve/pii/S1389128621005739>.
- Leo Breiman. Random Forests. *Machine Learning*, 45(1):5–32, October 2001. ISSN 0885-6125, 1573-0565. doi: 10.1023/A:1010933404324. URL <https://link.springer.com/10.1023/A:1010933404324>.
- Marco Cantone, Claudio Marrocco, and Alessandro Bria. Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study. *IEEE Access*, 12:144489–144508, 2024. ISSN 2169-3536. doi: 10.1109/ACCESS.2024.3472907. URL <https://ieeexplore.ieee.org/document/10704637/>.
- Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, October 2001. ISSN 0090-5364. doi: 10.1214/aos/1013203451. URL <https://doi.org/10.1214/aos/1013203451>.
- Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, December 2012. ISSN 1556-4681, 1556-472X. doi: 10.1145/2382577.2382579. URL <https://dl.acm.org/doi/10.1145/2382577.2382579>.
- Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In Isabelle Guyon, Ulrike Von Luxburg, Samy Bengio, Hanna Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html.
- NFStream contributors. NFStream apis documentation, s.d. URL <https://www.nfstream.org/docs/api>.
- Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd international conference on Machine learning - ICML '05*, pages 625–632, Bonn, Germany, 2005. ACM Press. ISBN 978-1-59593-180-1. doi: 10.1145/1102351.1102430. URL <https://dl.acm.org/doi/10.1145/1102351.1102430>.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vander-

- plas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- Karen Scarfone and Peter Mell. Guide to Intrusion Detection and Prevention Systems (IDPS). Technical Report NIST SP 800-94, National Institute of Standards and Technology, Gaithersburg, MD, 2007. URL <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-94.pdf>.
- scikit-learn developers. Probability calibration, s.d. URL <https://scikit-learn.org/stable/modules/calibration.html>.
- B. S. Sharmila and Rohini Nagapadma. RT-IoT2022, 2023a. URL <https://archive.ics.uci.edu/dataset/942>.
- B S Sharmila and Rohini Nagapadma. Quantized autoencoder (QAE) intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset. *Cybersecurity*, 6(1):41, September 2023b. ISSN 2523-3246. doi: 10.1186/s42400-023-00178-5. URL <https://cybersecurity.springeropen.com/articles/10.1186/s42400-023-00178-5>.
- Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, July 2009. ISSN 03064573. doi: 10.1016/j.ipm.2009.03.002. URL <https://linkinghub.elsevier.com/retrieve/pii/S0306457309000259>.
- Paul Watrobski, Murugiah Souppaya, Joshua Klosterman, and William Barker. Methodology for characterizing network behavior of internet of things devices. Technical Report NIST IR 8349, National Institute of Standards and Technology (U.S.), Gaithersburg, MD, August 2025. URL <https://nvlpubs.nist.gov/nistpubs/ir/2025/NIST.IR.8349.pdf>.
- Bruno Bogaz Zarpelão, Rodrigo Sanches Miani, Cláudio Toshio Kawakani, and Sean Carlito De Alvarenga. A survey of intrusion detection in Internet of Things. *Journal of Network and Computer Applications*, 84:25–37, April 2017. ISSN 10848045. doi: 10.1016/j.jnca.2017.02.009. URL <https://linkinghub.elsevier.com/retrieve/pii/S1084804517300802>.